

Top 75 LeetCode Questions — Beginner-Friendly Walkthroughs

Plus patterns reported in public ride-hailing / latency-sensitive interview loops

Version compiled 2026-05-05.

Personal study notes. This guide aggregates well-known algorithmic patterns and classic interview problems for my own prep. Problem names mirror LeetCode's conventions for navigability; problem statements are paraphrased rather than copied. The "company-specific" section is sourced from publicly-shared interview reports on Glassdoor and LeetCode Discuss. **Not affiliated with or endorsed by LeetCode, Uber, or any other company mentioned.**

How to use this guide

Each problem follows the same recipe so the muscle becomes automatic in interviews:

1. **Restate the problem in your own words.**
2. **Ask 3 clarifying questions** before touching any whiteboard. Examples: - "Are inputs sorted?" "Can the array be empty?" "Are there duplicates?" "What's the value range?" "Should I optimize for time, space, or both?"
3. **Walk through a tiny example by hand.** This is your debugging anchor.
4. **State the brute force first** with its complexity. Even if it times out, it shows you understand the problem.
5. **Identify the bottleneck** in the brute force ("we re-scan the same prefix N times...").
6. **Propose the optimization** in plain English. Only then write code.
7. **Trace your code on the example** to catch off-by-ones.
8. **State the final time and space complexity** without prompting.

Notation: `n` is the input size, `m` for second-dimension, `k` for query/k-th. Code is Python 3 unless stated.

The patterns cheat sheet

Pattern	Trigger phrase	Typical complexity
Hash map / set	"find duplicates", "two-sum-style", "frequency"	$O(n)$ time
Two pointers	"sorted array", "in-place", "pair / triplet"	$O(n)$ or $O(n^2)$
Sliding window	"longest / shortest substring or subarray with property"	$O(n)$
Binary search	"sorted", "minimum value such that...", "rotated"	$O(\log n)$

Pattern	Trigger phrase	Typical complexity
BFS	"shortest path on unweighted graph / grid"	$O(V+E)$
DFS / backtracking	"all combinations / permutations / paths"	exponential
Topological sort	"course / task ordering", "dependency"	$O(V+E)$
Union-find	"connectivity", "merging groups"	$\sim O(\alpha(n))$ per op
Heap	"k-th largest / smallest", "running median"	$O(n \log k)$
Trie	"prefix lookup", "autocomplete"	$O(L)$ per op
Greedy	"intervals", "scheduling", "minimum jumps"	$O(n \log n)$
DP 1D	"min/max ending at i", "ways to reach n"	$O(n)$
DP 2D	"two strings / two pointers in DP"	$O(nm)$
Monotonic stack	"next greater / smaller element"	$O(n)$

Part I — Arrays & Hashing

1. Two Sum — LC 1 · Easy

Problem

Given an array `nums` and a target `t`, return the indices of the two numbers that add up to `t`. Exactly one solution exists. You may not use the same element twice.

Clarifying questions

- Are there duplicate values? (Yes — but indices are still unique.)
- Is the array sorted? (No, generally not.)
- Negative numbers allowed? (Yes.)
- Return indices or values? (Indices.)

Brute force

Two nested loops. For each `i`, scan `j > i` and check `nums[i] + nums[j] == t`. *Time:* $O(n^2)$ · *Space:* $O(1)$

Optimal — hash map of seen values

Walk once. For each `x`, the partner is `t - x`. If we've seen the partner, we're done.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        partner = target - x
        if partner in seen:
            return [seen[partner], i]
        seen[x] = i
    return []
```

Time: $O(n)$ · Space: $O(n)$

Walkthrough

`nums = [2, 7, 11, 15]`, `t = 9`. At `i=0`: store `2→0`. At `i=1`: partner=2, found at 0 → `[0, 1]`.

Variations

LC 167 — Two Sum II (sorted input). Two pointers from both ends; move based on sum vs target.

```
def two_sum_sorted(nums, target):
    l, r = 0, len(nums) - 1
    while l < r:
        s = nums[l] + nums[r]
        if s == target: return [l + 1, r + 1] # 1-indexed
        if s < target: l += 1
        else: r -= 1
```

Time: $O(n)$ · Space: $O(1)$

LC 653 — Two Sum on BST. In-order yields a sorted array; then two pointers (or use a set during traversal).

```
def two_sum_bst(root, k):
    seen = set()
    def dfs(n):
        if not n: return False
        if k - n.val in seen: return True
        seen.add(n.val)
        return dfs(n.left) or dfs(n.right)
    return dfs(root)
```

Time: $O(n)$ · Space: $O(n)$

LC 18 — 4Sum. Two outer loops + two pointers, with dedup at every level.

```
def four_sum(nums, target):
    nums.sort()
    n, out = len(nums), []
    for i in range(n - 3):
        if i > 0 and nums[i] == nums[i-1]: continue
        for j in range(i + 1, n - 2):
            if j > i + 1 and nums[j] == nums[j-1]: continue
            l, r = j + 1, n - 1
            while l < r:
                s = nums[i] + nums[j] + nums[l] + nums[r]
                if s == target:
                    out.append([nums[i], nums[j], nums[l], nums[r]])
                    l += 1; r -= 1
                    while l < r and nums[l] == nums[l-1]: l += 1
                    while l < r and nums[r] == nums[r+1]: r -= 1
                elif s < target: l += 1
                else: r -= 1
    return out
```

Time: $O(n^3)$ · Space: $O(1)$ extra

2. Best Time to Buy and Sell Stock — LC 121 · Easy

Problem

Given daily prices, find the max profit from a single buy-then-sell. You must buy before you sell.

Clarifying questions

- Can profit be 0? (Yes if prices are non-increasing.)
- Empty input? (Return 0.)
- Same-day buy/sell allowed? (No — buy day must be earlier.)

Brute force

Try every (i, j) with $i < j$. Time: $O(n^2)$ · Space: $O(1)$

Optimal — track min so far

Keep `min_price`; candidate profit each day is `price - min_price`.

```
def max_profit(prices):
    min_price = float('inf')
    best = 0
    for p in prices:
        min_price = min(min_price, p)
        best = max(best, p - min_price)
    return best
```

Time: $O(n)$ · Space: $O(1)$

Walkthrough

[7, 1, 5, 3, 6, 4]. min=1 from day 2. Best profit = 6 - 1 = 5.

Variations

LC 122 — Multiple transactions allowed. Sum every positive day-to-day delta; that's the optimum.

```
def max_profit_unlimited(prices):  
    return sum(max(0, prices[i] - prices[i-1]) for i in range(1, len(prices)))
```

Time: $O(n)$ · Space: $O(1)$

LC 123 — At most 2 transactions. Track four states across the array.

```
def max_profit_two(prices):  
    buy1 = buy2 = float('-inf')  
    sell1 = sell2 = 0  
    for p in prices:  
        buy1 = max(buy1, -p)  
        sell1 = max(sell1, buy1 + p)  
        buy2 = max(buy2, sell1 - p)  
        sell2 = max(sell2, buy2 + p)  
    return sell2
```

Time: $O(n)$ · Space: $O(1)$

LC 188 — At most k transactions. Generalize the four-state to $2k$ states.

```
def max_profit_k(k, prices):  
    n = len(prices)  
    if k >= n // 2: # acts like unlimited  
        return sum(max(0, prices[i] - prices[i-1]) for i in range(1, n))  
    buy = [float('-inf')] * (k + 1)  
    sell = [0] * (k + 1)  
    for p in prices:  
        for j in range(1, k + 1):  
            buy[j] = max(buy[j], sell[j-1] - p)  
            sell[j] = max(sell[j], buy[j] + p)  
    return sell[k]
```

Time: $O(nk)$ · Space: $O(k)$

LC 309 — With cooldown (1 day). Three rolling states.

```
def max_profit_cooldown(prices):  
    hold = float('-inf')  
    sold = 0  
    rest = 0  
    for p in prices:  
        prev_sold = sold  
        sold = hold + p  
        hold = max(hold, rest - p)  
        rest = max(rest, prev_sold)  
    return max(sold, rest)
```

Time: $O(n)$ · Space: $O(1)$

3. Contains Duplicate — LC 217 · Easy

Problem

Return true if any value appears at least twice.

Clarifying questions

- Empty array? (False.)
- Range of values? (Affects bitset alternative if extreme.)

Brute force

Sort and check adjacent pairs. *Time:* $O(n \log n)$ · *Space:* $O(1)$

Optimal — set

```
def contains_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False
```

Time: $O(n)$ · *Space:* $O(n)$

Variations

LC 219 — Duplicate within distance k. Sliding-window set of size k.

```
def contains_nearby_duplicate(nums, k):
    window = set()
    for i, x in enumerate(nums):
        if x in window: return True
        window.add(x)
        if len(window) > k:
            window.remove(nums[i - k])
    return False
```

Time: $O(n)$ · *Space:* $O(k)$

LC 220 — Within k indices, abs diff $\leq t$. Bucketize by $t+1$; collisions inside or between adjacent buckets.

```
def contains_nearby_almost(nums, k, t):
    if t < 0: return False
    buckets = {}
    w = t + 1
    for i, x in enumerate(nums):
        b = x // w
        if b in buckets: return True
        if b - 1 in buckets and abs(x - buckets[b - 1]) < w: return True
        if b + 1 in buckets and abs(x - buckets[b + 1]) < w: return True
        buckets[b] = x
        if i >= k:
            del buckets[nums[i - k] // w]
    return False
```

Time: $O(n)$ · Space: $O(k)$

4. Product of Array Except Self — LC 238 · Medium

Problem

Return `out[i] = product of all nums except nums[i]`. No division. $O(n)$ time.

Clarifying questions

- Are zeros possible? (Yes.)
- Should the answer fit in 32-bit int? (Problem guarantees.)
- Output array counts as space? (Conventionally not.)

Brute force

For each i , multiply all others. *Time: $O(n^2)$ · Space: $O(1)$*

Optimal — left-prefix then right-suffix

```
def product_except_self(nums):
    n = len(nums)
    out = [1] * n
    left = 1
    for i in range(n):
        out[i] = left
        left *= nums[i]
    right = 1
    for i in range(n - 1, -1, -1):
        out[i] *= right
        right *= nums[i]
    return out
```

Time: $O(n)$ · Space: $O(1)$ extra

Walkthrough

[1,2,3,4] . After left pass: [1,1,2,6] . After right pass: [24,12,8,6] .

Variations

Zero-aware variant (interview twist). Count zeros and total product of non-zeros.

```
def product_with_zeros(nums):
    n = len(nums)
    zeros = nums.count(0)
    if zeros >= 2:
        return [0] * n
    if zeros == 1:
        prod = 1
        for x in nums:
            if x != 0: prod *= x
        return [prod if x == 0 else 0 for x in nums]
    total = 1
    for x in nums: total *= x
    return [total // x for x in nums]    # OK because no zeros
```

Time: $O(n)$ · Space: $O(1)$

LC 1352 — Product of last K (with rollback). Maintain prefix products; on a 0 reset.

```
class ProductOfNumbers:
    def __init__(self):
        self.pre = [1]
    def add(self, x):
        if x == 0:
            self.pre = [1]
        else:
            self.pre.append(self.pre[-1] * x)
    def getProduct(self, k):
        if k >= len(self.pre): return 0
        return self.pre[-1] // self.pre[-1 - k]
```

Time: $O(1)$ per op · Space: $O(n)$

5. Maximum Subarray — LC 53 · Medium

Problem

Find the contiguous subarray with the largest sum.

Clarifying questions

- All negative numbers possible? (Yes — return the largest single element.)
- Need indices, or just sum? (Just sum unless asked.)

Brute force

Every (i, j) pair. *Time:* $O(n^2)$ · *Space:* $O(1)$

Optimal — Kadane's algorithm

```
def max_sub_array(nums):
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

Time: $O(n)$ · *Space:* $O(1)$

Walkthrough

`[-2, 1, -3, 4, -1, 2, 1, -5, 4]` → best = 6 (subarray `[4, -1, 2, 1]`).

Variations

Return the actual subarray.

```
def max_subarray_indices(nums):
    cur_start = best_start = best_end = 0
    cur = best = nums[0]
    for i in range(1, len(nums)):
        if nums[i] > cur + nums[i]:
            cur = nums[i]
            cur_start = i
        else:
            cur += nums[i]
        if cur > best:
            best = cur
            best_start, best_end = cur_start, i
    return nums[best_start:best_end + 1]
```

Time: $O(n)$ · *Space:* $O(1)$

LC 918 — Max sum of circular subarray. `max(kadane_max, total - kadane_min)` unless the array is all-negative.

```
def max_subarray_sum_circular(nums):
    total = max_sum = cur_max = nums[0]
    min_sum = cur_min = nums[0]
    for x in nums[1:]:
        cur_max = max(x, cur_max + x)
        max_sum = max(max_sum, cur_max)
        cur_min = min(x, cur_min + x)
        min_sum = min(min_sum, cur_min)
        total += x
    return max_sum if max_sum < 0 else max(max_sum, total - min_sum)
```

Time: $O(n)$ · *Space:* $O(1)$

LC 1186 — Delete at most one element. Two states: best ending here without/with a deletion.

```
def maximum_sum(arr):
    no_del = with_del = arr[0]
    best = arr[0]
    for x in arr[1:]:
        with_del = max(no_del, with_del + x)
        no_del = max(x, no_del + x)
        best = max(best, no_del, with_del)
    return best
```

Time: $O(n)$ · Space: $O(1)$

6. Maximum Product Subarray — LC 152 · Medium

Problem

Largest product among all contiguous subarrays.

Clarifying questions

- Zeros and negatives present? (Yes — both flip dynamics.)
- Empty input? (Per LC, no.)

Brute force

All pairs. *Time: $O(n^2)$ · Space: $O(1)$*

Optimal — track running max and min

```
def max_product(nums):
    cur_max = cur_min = best = nums[0]
    for x in nums[1:]:
        cands = (x, cur_max * x, cur_min * x)
        cur_max = max(cands)
        cur_min = min(cands)
        best = max(best, cur_max)
    return best
```

Time: $O(n)$ · Space: $O(1)$

Variations

Return the actual subarray. Carry start/end indices alongside `cur_max` / `cur_min`.

```
def max_product_subarray(nums):
    cur_max = cur_min = nums[0]
    s_max = e_max = 0
    s_min = e_min = 0
    best = nums[0]; bs = be = 0
    for i in range(1, len(nums)):
        x = nums[i]
        if x < 0:
            cur_max, cur_min = cur_min, cur_max
            s_max, s_min = s_min, s_max
        if cur_max * x > x:
            cur_max *= x
            e_max = i
        else:
            cur_max = x; s_max = e_max = i
        if cur_min * x < x:
            cur_min *= x
            e_min = i
        else:
            cur_min = x; s_min = e_min = i
        if cur_max > best:
            best = cur_max; bs, be = s_max, e_max
    return nums[bs:be + 1]
```

Time: $O(n)$ · Space: $O(1)$

Subarray with at-most-one zero allowed. DP with extra state for "have we used the zero".

```
def max_product_one_zero(nums):
    n = len(nums)
    no = [(0, 0)] * n # (max, min) without using zero swap
    best = nums[0]
    for i, x in enumerate(nums):
        if i == 0:
            no[i] = (x, x)
            continue
        prev_max, prev_min = no[i-1]
        cand = [x, prev_max * x, prev_min * x]
        no[i] = (max(cand), min(cand))
        best = max(best, no[i][0])
    return best
```

Time: $O(n)$ · Space: $O(n)$

7. Find Minimum in Rotated Sorted Array — LC 153 · Medium

Problem

Sorted ascending array rotated at unknown pivot. No duplicates. Find the minimum.

Brute force

Linear scan. Time: $O(n)$ · Space: $O(1)$

Optimal — binary search comparing mid to right

```
def find_min(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[hi]: lo = mid + 1
        else: hi = mid
    return nums[lo]
```

Time: $O(\log n)$ · Space: $O(1)$

Variations

LC 154 — With duplicates. When `nums[mid] == nums[hi]`, `hi -= 1` (worst case $O(n)$).

```
def find_min_dup(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[hi]: lo = mid + 1
        elif nums[mid] < nums[hi]: hi = mid
        else: hi -= 1
    return nums[lo]
```

Time: $O(\log n)$ average, $O(n)$ worst · Space: $O(1)$

Find pivot index instead of value.

```
def find_pivot_index(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[hi]: lo = mid + 1
        else: hi = mid
    return lo
```

Time: $O(\log n)$ · Space: $O(1)$

8. Search in Rotated Sorted Array — LC 33 · Medium

Problem

Sorted array rotated at unknown pivot. Find target's index in $O(\log n)$ or return -1.

Brute force

Linear scan. Time: $O(n)$ · Space: $O(1)$

Optimal — binary search with "which half is sorted?"

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target: return mid
        if nums[lo] <= nums[mid]:
            if nums[lo] <= target < nums[mid]: hi = mid - 1
            else: lo = mid + 1
        else:
            if nums[mid] < target <= nums[hi]: lo = mid + 1
            else: hi = mid - 1
    return -1
```

Time: $O(\log n)$ · Space: $O(1)$

Variations

LC 81 — With duplicates. When `nums[lo] == nums[mid] == nums[hi]`, shrink both sides by one.

```
def search_dup(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target: return True
        if nums[lo] == nums[mid] == nums[hi]:
            lo += 1; hi -= 1
        elif nums[lo] <= nums[mid]:
            if nums[lo] <= target < nums[mid]: hi = mid - 1
            else: lo = mid + 1
        else:
            if nums[mid] < target <= nums[hi]: lo = mid + 1
            else: hi = mid - 1
    return False
```

Time: $O(\log n)$ average, $O(n)$ worst · Space: $O(1)$

LC 34 — Find leftmost / rightmost occurrence. Two binary searches.

```
from bisect import bisect_left, bisect_right
def search_range(nums, target):
    l = bisect_left(nums, target)
    if l == len(nums) or nums[l] != target: return [-1, -1]
    return [l, bisect_right(nums, target) - 1]
```

Time: $O(\log n)$ · Space: $O(1)$

Part II — Two Pointers

9. 3Sum — LC 15 · Medium

Problem

Return all unique triplets summing to zero.

Brute force

Three loops + set dedup. *Time:* $O(n^3)$ · *Space:* $O(n)$

Optimal — sort + fix one + two pointers

```
def three_sum(nums):
    nums.sort()
    out = []
    n = len(nums)
    for i in range(n - 2):
        if i > 0 and nums[i] == nums[i - 1]: continue
        if nums[i] > 0: break
        l, r = i + 1, n - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s == 0:
                out.append([nums[i], nums[l], nums[r]])
                l += 1; r -= 1
                while l < r and nums[l] == nums[l - 1]: l += 1
                while l < r and nums[r] == nums[r + 1]: r -= 1
            elif s < 0: l += 1
            else: r -= 1
    return out
```

Time: $O(n^2)$ · *Space:* $O(1)$ extra

Variations

LC 16 — 3Sum closest. Track best `|sum - target|`.

```
def three_sum_closest(nums, target):
    nums.sort()
    best = sum(nums[:3])
    for i in range(len(nums) - 2):
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if abs(s - target) < abs(best - target):
                best = s
            if s < target: l += 1
            else: r -= 1
    return best
```

Time: $O(n^2)$ · *Space:* $O(1)$

LC 259 — 3Sum smaller. Count triplets with $\text{sum} < \text{target}$. When found, every $r' \leq r$ works $\rightarrow$ add $r - l$.

```
def three_sum_smaller(nums, target):
    nums.sort()
    count = 0
    for i in range(len(nums) - 2):
        l, r = i + 1, len(nums) - 1
        while l < r:
            if nums[i] + nums[l] + nums[r] < target:
                count += r - l
                l += 1
            else:
                r -= 1
    return count
```

Time: $O(n^2)$ · *Space:* $O(1)$

10. Container With Most Water — LC 11 · Medium

Problem

Pick two vertical lines from heights to form a container; maximize water.

Brute force

All pairs. *Time:* $O(n^2)$ · *Space:* $O(1)$

Optimal — two pointers from the ends

```
def max_area(height):
    l, r = 0, len(height) - 1
    best = 0
    while l < r:
        h = min(height[l], height[r])
        best = max(best, h * (r - l))
        if height[l] < height[r]: l += 1
        else: r -= 1
    return best
```

Time: $O(n)$ · *Space:* $O(1)$

Variations

LC 42 — Trapping Rain Water (two-pointer form). See U2.

```
def trap(h):
    l, r = 0, len(h) - 1
    maxL = maxR = total = 0
    while l < r:
        if h[l] < h[r]:
            maxL = max(maxL, h[l]); total += maxL - h[l]; l += 1
        else:
            maxR = max(maxR, h[r]); total += maxR - h[r]; r -= 1
    return total
```

Time: $O(n)$ · Space: $O(1)$

11. Valid Palindrome — LC 125 · Easy

Problem

Considering only alphanumeric and ignoring case, is the string a palindrome?

Brute force

Filter + reverse compare. Time: $O(n)$ · Space: $O(n)$

Optimal — two pointers in place

```
def is_palindrome(s):
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum(): l += 1
        while l < r and not s[r].isalnum(): r -= 1
        if s[l].lower() != s[r].lower(): return False
        l += 1; r -= 1
    return True
```

Time: $O(n)$ · Space: $O(1)$

Variations

LC 680 — Delete at most one character. On mismatch, recursively try skipping either side.

```
def valid_palindrome_one_delete(s):
    def is_pal(l, r):
        while l < r:
            if s[l] != s[r]: return False
            l += 1; r -= 1
        return True
    l, r = 0, len(s) - 1
    while l < r:
        if s[l] != s[r]:
            return is_pal(l + 1, r) or is_pal(l, r - 1)
        l += 1; r -= 1
    return True
```

Time: $O(n)$ · Space: $O(1)$

LC 1216 — At most k deletions. Reduce to longest palindromic subsequence (LPS); answer = $(n - \text{LPS}) \leq k$.

```
def is_valid_palindrome_k(s, k):
    n = len(s)
    dp = [[0]*n for _ in range(n)]
    for i in range(n): dp[i][i] = 1
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j]:
                dp[i][j] = dp[i+1][j-1] + 2 if length > 2 else 2
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])
    return n - dp[0][n-1] <= k
```

Time: $O(n^2)$ · Space: $O(n^2)$

Part III — Sliding Window

12. Longest Substring Without Repeating Characters — LC 3 · Medium

Problem

Length of the longest substring with all distinct characters.

Brute force

Try every starting index. *Time: $O(n^2)$ · Space: $O(\sigma)$*

Optimal — sliding window with last-seen index

```
def length_of_longest_substring(s):
    last = {}
    l = 0
    best = 0
    for r, c in enumerate(s):
        if c in last and last[c] >= l:
            l = last[c] + 1
        last[c] = r
        best = max(best, r - l + 1)
    return best
```

Time: $O(n)$ · Space: $O(\sigma)$

Walkthrough

`s="abcabcbcb"` . Best window is `"abc"` of length 3.

Variations

LC 159 — At most 2 distinct characters. Generic "at-most-k" template, $k=2$.

```
def length_of_longest_substring_two_distinct(s):  
    return at_most_k_distinct(s, 2)
```

LC 340 — At most k distinct characters.

```
def at_most_k_distinct(s, k):  
    count = {}  
    l = best = 0  
    for r, c in enumerate(s):  
        count[c] = count.get(c, 0) + 1  
        while len(count) > k:  
            count[s[l]] -= 1  
            if count[s[l]] == 0: del count[s[l]]  
            l += 1  
        best = max(best, r - l + 1)  
    return best
```

Time: $O(n)$ · Space: $O(k)$

LC 992 — Subarrays with exactly K distinct. `atMost(k) - atMost(k - 1)` .

```
def subarrays_with_k_distinct(nums, k):  
    def at_most(K):  
        count = {}; l = total = 0  
        for r, x in enumerate(nums):  
            count[x] = count.get(x, 0) + 1  
            while len(count) > K:  
                count[nums[l]] -= 1  
                if count[nums[l]] == 0: del count[nums[l]]  
                l += 1  
            total += r - l + 1  
        return total  
    return at_most(k) - at_most(k - 1)
```

Time: $O(n)$ · Space: $O(n)$

13. Longest Repeating Character Replacement — LC 424 · Medium

Problem

Longest substring you can make all-same by replacing at most `k` characters.

Brute force

Every (l, r), check $(r - l + 1) - \text{max_freq} \leq k$. *Time: $O(n^2)$ · Space: $O(\sigma)$*

Optimal — sliding window

Window valid when $\text{window_size} - \text{max_freq} \leq k$.

```
def character_replacement(s, k):
    count = {}
    l = max_freq = best = 0
    for r, c in enumerate(s):
        count[c] = count.get(c, 0) + 1
        max_freq = max(max_freq, count[c])
        while (r - l + 1) - max_freq > k:
            count[s[l]] -= 1
            l += 1
        best = max(best, r - l + 1)
    return best
```

Time: $O(n)$ · Space: $O(\sigma)$

Variations

LC 1004 — Max consecutive ones III (flip $\leq k$ zeros). Same template, count zeros.

```
def longest_ones(nums, k):
    l = zeros = best = 0
    for r, x in enumerate(nums):
        if x == 0: zeros += 1
        while zeros > k:
            if nums[l] == 0: zeros -= 1
            l += 1
        best = max(best, r - l + 1)
    return best
```

Time: $O(n)$ · Space: $O(1)$

LC 1493 — Longest subarray of 1s after deleting one element. Sliding window with at-most-one zero.

```
def longest_subarray_after_one_delete(nums):
    l = zeros = best = 0
    for r, x in enumerate(nums):
        if x == 0: zeros += 1
        while zeros > 1:
            if nums[l] == 0: zeros -= 1
            l += 1
        best = max(best, r - l)
    return best
```

Time: $O(n)$ · Space: $O(1)$

14. Minimum Window Substring — LC 76 · Hard

Problem

Minimum window in `s` that contains every character of `t` (with multiplicity).

Brute force

Every substring + count check. *Time:* $O(n^2 \cdot \sigma)$ *Space:* $O(\sigma)$

Optimal — sliding window with `have / need`

```
from collections import Counter

def min_window(s, t):
    if not s or not t: return ""
    need = Counter(t)
    need_distinct = len(need)
    have = 0
    counts = {}
    best = (float('inf'), 0, 0)
    l = 0
    for r, c in enumerate(s):
        counts[c] = counts.get(c, 0) + 1
        if c in need and counts[c] == need[c]:
            have += 1
        while have == need_distinct:
            if r - l + 1 < best[0]:
                best = (r - l + 1, l, r)
            counts[s[l]] -= 1
            if s[l] in need and counts[s[l]] < need[s[l]]:
                have -= 1
            l += 1
    return "" if best[0] == float('inf') else s[best[1]:best[2] + 1]
```

Time: $O(n + m)$ *Space:* $O(\sigma)$

Variations

LC 567 — Permutation in string. Fixed-size window of `len(t)` compared to target counts.

```
def check_inclusion(t, s):
    if len(t) > len(s): return False
    need = [0]*26; have = [0]*26
    a = ord('a')
    for c in t: need[ord(c)-a] += 1
    for i, c in enumerate(s):
        have[ord(c)-a] += 1
        if i >= len(t):
            have[ord(s[i-len(t)])-a] -= 1
        if have == need: return True
    return False
```

Time: $O(n)$ *Space:* $O(1)$

LC 438 — Find all anagrams of p in s.

```
def find_anagrams(s, p):
    if len(p) > len(s): return []
    need = [0]*26; have = [0]*26
    a = ord('a')
    for c in p: need[ord(c)-a] += 1
    out = []
    for i, c in enumerate(s):
        have[ord(c)-a] += 1
        if i >= len(p):
            have[ord(s[i-len(p)])-a] -= 1
            if have == need:
                out.append(i - len(p) + 1)
    return out
```

Time: $O(n)$ · Space: $O(1)$

Part IV — Strings

15. Valid Anagram — LC 242 · Easy

Problem

Are `s` and `t` anagrams (same letters, possibly reordered)?

Brute force

Sort both, compare. Time: $O(n \log n)$ · Space: $O(n)$

Optimal — count letters

```
def is_anagram(s, t):
    if len(s) != len(t): return False
    cnt = {}
    for c in s:
        cnt[c] = cnt.get(c, 0) + 1
    for c in t:
        if cnt.get(c, 0) == 0: return False
        cnt[c] -= 1
    return True
```

Time: $O(n)$ · Space: $O(\sigma)$

Variations

Anagram with unicode. Use `defaultdict(int)`.

```

from collections import defaultdict
def is_anagram_unicode(s, t):
    if len(s) != len(t): return False
    cnt = defaultdict(int)
    for c in s: cnt[c] += 1
    for c in t:
        cnt[c] -= 1
        if cnt[c] < 0: return False
    return True

```

Time: $O(n)$ · Space: $O(\sigma)$

Streaming compare (one pass on each). Returns True/False with bounded memory if charset is bounded.

16. Group Anagrams — LC 49 · Medium

Problem

Group strings that are anagrams of each other.

Brute force

For every pair, check. *Time: $O(n^2 \cdot k)$ · Space: $O(n)$*

Optimal — group by canonical form

```

from collections import defaultdict

def group_anagrams(strs):
    out = defaultdict(list)
    for s in strs:
        key = tuple(sorted(s))
        out[key].append(s)
    return list(out.values())

```

Time: $O(n \cdot k \log k)$ · Space: $O(n \cdot k)$

Variations

Count-tuple key (faster for ASCII).

```

def group_anagrams_count(strs):
    out = defaultdict(list)
    for s in strs:
        key = [0]*26
        for c in s: key[ord(c)-ord('a')] += 1
        out[tuple(key)].append(s)
    return list(out.values())

```

Time: $O(n \cdot k)$ · Space: $O(n \cdot k)$

Streaming version (online). Maintain dict on each insert; same data structure.

17. Valid Parentheses — LC 20 · Easy

Problem

Given a string of `()[]{}` , is it balanced?

Brute force

Repeatedly strip empty pairs. *Time:* $O(n^2)$ · *Space:* $O(n)$

Optimal — stack

```
def is_valid(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for c in s:
        if c in pairs:
            if not stack or stack[-1] != pairs[c]: return False
            stack.pop()
        else:
            stack.append(c)
    return not stack
```

Time: $O(n)$ · *Space:* $O(n)$

Variations

LC 32 — Longest valid parentheses. Stack of indices.

```
def longest_valid_parentheses(s):
    stack = [-1]
    best = 0
    for i, c in enumerate(s):
        if c == '(':
            stack.append(i)
        else:
            stack.pop()
            if not stack:
                stack.append(i)
            else:
                best = max(best, i - stack[-1])
    return best
```

Time: $O(n)$ · *Space:* $O(n)$

LC 921 — Min add to make valid. Counter sweep — track unmatched `(` and `)`.

```
def min_add_to_make_valid(s):
    open_unmatched = close_unmatched = 0
    for c in s:
        if c == '(':
            open_unmatched += 1
        else:
            if open_unmatched > 0: open_unmatched -= 1
            else: close_unmatched += 1
    return open_unmatched + close_unmatched
```

Time: O(n) · Space: O(1)

LC 1249 — Min remove to make valid (return string). Two-pass mark with stack.

```
def min_remove_to_make_valid(s):
    chars = list(s); stack = []
    for i, c in enumerate(chars):
        if c == '(':
            stack.append(i)
        elif c == ')':
            if stack: stack.pop()
            else: chars[i] = ''
    for i in stack: chars[i] = ''
    return ''.join(chars)
```

Time: O(n) · Space: O(n)

18. Longest Palindromic Substring — LC 5 · Medium

Problem

Find the longest substring that is a palindrome.

Brute force

Every substring + palindrome check. *Time: O(n³) · Space: O(1)*

Optimal — expand around centers

```
def longest_palindrome(s):
    if not s: return ""
    start = end = 0
    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1; r += 1
        return l + 1, r - 1
    for i in range(len(s)):
        l1, r1 = expand(i, i)
        l2, r2 = expand(i, i + 1)
        if r1 - l1 > end - start: start, end = l1, r1
        if r2 - l2 > end - start: start, end = l2, r2
    return s[start:end + 1]
```

Time: $O(n^2)$ · Space: $O(1)$

Variations

LC 647 — Count palindromic substrings. Same expansion; count each.

```
def count_substrings(s):
    n = len(s); total = 0
    def expand(l, r):
        nonlocal total
        while l >= 0 and r < n and s[l] == s[r]:
            total += 1; l -= 1; r += 1
    for i in range(n):
        expand(i, i); expand(i, i + 1)
    return total
```

Time: $O(n^2)$ · Space: $O(1)$

Manacher's algorithm — $O(n)$. Worth knowing exists; rare to be asked to implement.

```
def manacher(s):
    t = '#' + '#'.join(s) + '#'
    p = [0]*len(t)
    c = r = 0
    for i in range(len(t)):
        mirror = 2*c - i
        if i < r: p[i] = min(r - i, p[mirror])
        while i - p[i] - 1 >= 0 and i + p[i] + 1 < len(t) and t[i - p[i] - 1] == t[i + p[i] + 1]:
            p[i] += 1
        if i + p[i] > r:
            c = i; r = i + p[i]
    k = max(range(len(t)), key=lambda i: p[i])
    return s[(k - p[k])//2 : (k + p[k])//2]
```

Time: $O(n)$ · Space: $O(n)$

19. Palindromic Substrings — LC 647 · Medium

Problem

Count all palindromic substrings (overlapping allowed).

Optimal — same expansion as above

```
def count_substrings(s):
    n = len(s); total = 0
    def expand(l, r):
        nonlocal total
        while l >= 0 and r < n and s[l] == s[r]:
            total += 1; l -= 1; r += 1
    for i in range(n):
        expand(i, i); expand(i, i + 1)
    return total
```

Time: $O(n^2)$ · Space: $O(1)$

Variations

DP version (clearer for some).

```
def count_substrings_dp(s):
    n = len(s); total = 0
    dp = [[False]*n for _ in range(n)]
    for length in range(1, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j] and (length < 3 or dp[i+1][j-1]):
                dp[i][j] = True
                total += 1
    return total
```

Time: $O(n^2)$ · Space: $O(n^2)$

Longest palindromic subsequence (different problem, LC 516). Classic 2D DP.

```
def longest_palindrome_subseq(s):
    n = len(s)
    dp = [[0]*n for _ in range(n)]
    for i in range(n): dp[i][i] = 1
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j]:
                dp[i][j] = (dp[i+1][j-1] if length > 2 else 0) + 2
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])
    return dp[0][n-1]
```

Time: $O(n^2)$ · Space: $O(n^2)$

20. Encode and Decode Strings — LC 271 · Medium (LC Premium)

Problem

Design `encode(list[str]) -> str` and `decode(str) -> list[str]` that round-trip.

Optimal — length-prefix framing

```
def encode(words):
    return ''.join(f"{len(w)}#{w}" for w in words)

def decode(s):
    out = []; i = 0
    while i < len(s):
        j = s.index('#', i)
        n = int(s[i:j])
        out.append(s[j + 1:j + 1 + n])
        i = j + 1 + n
    return out
```

Time: $O(\text{total chars})$ · *Space:* $O(\text{total chars})$

Variations

Delimiter-only (broken on inputs containing the delimiter) — anti-pattern but interview-relevant.

```
def encode_naive(words, sep='\x00'):
    if any(sep in w for w in words):
        raise ValueError('input contains separator')
    return sep.join(words)

def decode_naive(s, sep='\x00'):
    return s.split(sep)
```

Time: $O(n)$ · *Space:* $O(n)$

Streaming codec (unbounded list). Same length-prefix pattern, written incrementally.

```
class Codec:
    def __init__(self):
        self.buf = []
    def push(self, w):
        self.buf.append(f"{len(w)}#{w}")
    def all(self):
        return ''.join(self.buf)
```

Time: $O(1)$ per push · *Space:* $O(n)$

Part V — Bit Manipulation

21. Sum of Two Integers — LC 371 · Medium

Problem

Add two integers without using `+` or `-`.

Optimal — XOR + AND

```
def get_sum(a, b):
    MASK = 0xFFFFFFFF
    INT_MAX = 0x7FFFFFFF
    while b & MASK:
        carry = ((a & b) << 1) & MASK
        a = (a ^ b) & MASK
        b = carry
    return a if a <= INT_MAX else ~(a ^ MASK)
```

Time: O(1) (bounded by 32) · *Space:* O(1)

Variations

Subtract via bitwise. $a - b == a + (\sim b + 1)$ (twos complement).

```
def bit_sub(a, b):
    return get_sum(a, get_sum(~b, 1))
```

Time: O(1) · *Space:* O(1)

22. Number of 1 Bits — LC 191 · Easy

Problem

Count set bits in an unsigned integer.

Optimal — Brian Kernighan

```
def hamming_weight(n):
    count = 0
    while n:
        n &= n - 1
        count += 1
    return count
```

Time: O(k) where k = number of set bits · *Space:* O(1)

Variations

LC 461 — Hamming distance. popcount of XOR.

```
def hamming_distance(x, y):
    return bin(x ^ y).count('1')
```

Time: O(1) · *Space:* O(1)

LC 477 — Total Hamming distance for an array. Per bit position, count zeros and ones; contribution = $z \cdot o$.

```
def total_hamming_distance(nums):
    total = 0
    for b in range(32):
        ones = sum((x >> b) & 1 for x in nums)
        total += ones * (len(nums) - ones)
    return total
```

Time: $O(32n)$ · *Space:* $O(1)$

23. Counting Bits — LC 338 · Easy

Problem

For $0..n$, return `out[i] = popcount(i)`.

Brute force

LC 191 for each. *Time:* $O(n \log n)$ · *Space:* $O(n)$

Optimal — DP `out[i] = out[i >> 1] + (i & 1)`

```
def count_bits(n):
    out = [0] * (n + 1)
    for i in range(1, n + 1):
        out[i] = out[i >> 1] + (i & 1)
    return out
```

Time: $O(n)$ · *Space:* $O(n)$

Variations

Alternative recurrence: `out[i] = out[i & (i - 1)] + 1`.

```
def count_bits_kernighan(n):
    out = [0] * (n + 1)
    for i in range(1, n + 1):
        out[i] = out[i & (i - 1)] + 1
    return out
```

Time: $O(n)$ · *Space:* $O(n)$

24. Missing Number — LC 268 · Easy

Problem

Array contains n distinct numbers from $[0..n]$ with one missing. Find it.

Brute force

Sort and scan. *Time:* $O(n \log n)$ · *Space:* $O(1)$

Optimal — XOR or sum formula

```
def missing_number(nums):
    x = len(nums)
    for i, v in enumerate(nums):
        x ^= i ^ v
    return x
```

Time: $O(n)$ · *Space:* $O(1)$

Variations

Sum formula.

```
def missing_number_sum(nums):
    n = len(nums)
    return n*(n+1)//2 - sum(nums)
```

Time: $O(n)$ · *Space:* $O(1)$

LC 645 — Find duplicate AND missing. XOR all + bit grouping.

```
def find_dup_missing(nums):
    n = len(nums)
    x = 0
    for i, v in enumerate(nums):
        x ^= (i + 1) ^ v
    diff_bit = x & -x
    a = b = 0
    for i, v in enumerate(nums):
        if (i + 1) & diff_bit: a ^= (i + 1)
        else: b ^= (i + 1)
        if v & diff_bit: a ^= v
        else: b ^= v
    if a in nums: return [a, b]
    return [b, a]
```

Time: $O(n)$ · *Space:* $O(1)$

25. Reverse Bits — LC 190 · Easy

Problem

Reverse the bits of a 32-bit unsigned integer.

Optimal

```
def reverse_bits(n):
    r = 0
    for _ in range(32):
        r = (r << 1) | (n & 1)
        n >>= 1
    return r
```

Time: O(1) (32 iterations) · *Space:* O(1)

Variations

Optimized with byte-swap masks. Faster in compiled languages; mostly trivia in Python.

```
def reverse_bits_mask(n):
    n = ((n & 0xFFFF0000) >> 16) | ((n & 0x0000FFFF) << 16)
    n = ((n & 0xFF00FF00) >> 8) | ((n & 0x00FF00FF) << 8)
    n = ((n & 0xF0F0F0F0) >> 4) | ((n & 0x0F0F0F0F) << 4)
    n = ((n & 0xCCCCCCCC) >> 2) | ((n & 0x33333333) << 2)
    n = ((n & 0xAAAAAAAA) >> 1) | ((n & 0x55555555) << 1)
    return n
```

Time: O(1) · *Space:* O(1)

Reverse k bits.

```
def reverse_k_bits(n, k):
    r = 0
    for _ in range(k):
        r = (r << 1) | (n & 1)
        n >>= 1
    return r
```

Time: O(k) · *Space:* O(1)

Part VI — Linked List

```
Standard ListNode: python class ListNode: def __init__(self, val=0, next=None):
self.val = val self.next = next
```

26. Reverse Linked List — LC 206 · Easy

Problem

Reverse a singly linked list.

Iterative — three pointers

```
def reverse_list(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev = cur
        cur = nxt
    return prev
```

Time: $O(n)$ · Space: $O(1)$

Variations

Recursive.

```
def reverse_list_recursive(head):
    if not head or not head.next: return head
    new_head = reverse_list_recursive(head.next)
    head.next.next = head
    head.next = None
    return new_head
```

Time: $O(n)$ · Space: $O(n)$ recursion

LC 92 — Reverse between positions m and n . Walk to position before m , reverse the slice, reattach.

```
def reverse_between(head, m, n):
    dummy = ListNode(0, head)
    prev = dummy
    for _ in range(m - 1): prev = prev.next
    cur = prev.next
    for _ in range(n - m):
        nxt = cur.next
        cur.next = nxt.next
        nxt.next = prev.next
        prev.next = nxt
    return dummy.next
```

Time: $O(n)$ · Space: $O(1)$

LC 25 — Reverse k group. Chunk by k , reverse each chunk.

```
def reverse_k_group(head, k):
    # count first
    count = 0; cur = head
    while cur and count < k:
        cur = cur.next; count += 1
    if count < k: return head
    prev = None; cur = head
    for _ in range(k):
        nxt = cur.next; cur.next = prev; prev = cur; cur = nxt
    head.next = reverse_k_group(cur, k)
    return prev
```

Time: $O(n)$ · Space: $O(n/k)$ recursion

27. Linked List Cycle — LC 141 · Easy

Problem

Does the list contain a cycle?

Brute force

Hash set of visited nodes. Time: $O(n)$ · Space: $O(n)$

Optimal — Floyd's tortoise and hare

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast: return True
    return False
```

Time: $O(n)$ · Space: $O(1)$

Variations

LC 142 — Find cycle entry node. After collision, restart one pointer at head; advance both by 1.

```
def detect_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next; fast = fast.next.next
        if slow is fast:
            slow = head
            while slow is not fast:
                slow = slow.next; fast = fast.next
            return slow
    return None
```

Time: $O(n)$ · Space: $O(1)$

LC 287 — Find duplicate (Floyd on indexing).

```
def find_duplicate(nums):
    slow = fast = nums[0]
    while True:
        slow = nums[slow]; fast = nums[nums[fast]]
        if slow == fast: break
    slow = nums[0]
    while slow != fast:
        slow = nums[slow]; fast = nums[fast]
    return slow
```

Time: $O(n)$ · Space: $O(1)$

28. Merge Two Sorted Lists — LC 21 · Easy

Problem

Merge two sorted linked lists.

Optimal — dummy head

```
def merge_two_lists(a, b):
    dummy = ListNode(0); tail = dummy
    while a and b:
        if a.val <= b.val: tail.next = a; a = a.next
        else: tail.next = b; b = b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next
```

Time: $O(n + m)$ · Space: $O(1)$

Variations

Recursive merge.

```
def merge_two_recursive(a, b):
    if not a: return b
    if not b: return a
    if a.val <= b.val:
        a.next = merge_two_recursive(a.next, b)
        return a
    b.next = merge_two_recursive(a, b.next)
    return b
```

Time: $O(n + m)$ · Space: $O(n + m)$ recursion

LC 148 — Sort linked list (merge sort). Split with slow/fast, recurse, merge.

```
def sort_list(head):
    if not head or not head.next: return head
    slow = head; fast = head.next
    while fast and fast.next:
        slow = slow.next; fast = fast.next.next
    mid = slow.next; slow.next = None
    return merge_two_lists(sort_list(head), sort_list(mid))
```

Time: $O(n \log n)$ · *Space:* $O(\log n)$ recursion

29. Merge K Sorted Lists — LC 23 · Hard

Problem

Merge k sorted lists into one.

Brute force

Concat + sort. *Time:* $O(N \log N)$ · *Space:* $O(N)$

Optimal A — min-heap of heads

```
import heapq

def merge_k_lists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))
    dummy = ListNode(0); tail = dummy
    while heap:
        val, i, node = heapq.heappop(heap)
        tail.next = node; tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

Time: $O(N \log k)$ · *Space:* $O(k)$

Variations

Pairwise merge (no heap). Same complexity as heap.

```
def merge_k_lists_pairwise(lists):
    if not lists: return None
    while len(lists) > 1:
        merged = []
        for i in range(0, len(lists), 2):
            a = lists[i]
            b = lists[i+1] if i+1 < len(lists) else None
            merged.append(merge_two_lists(a, b))
        lists = merged
    return lists[0]
```

Time: $O(N \log k)$ · Space: $O(1)$

30. Remove Nth Node From End of List — LC 19 · Medium

Problem

Remove the n-th node from the end. Return new head.

Optimal — two pointers with gap n

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n + 1): fast = fast.next
    while fast:
        fast = fast.next; slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

Time: $O(L)$ · Space: $O(1)$

Variations

Two-pass (length first, then walk).

```
def remove_nth_two_pass(head, n):
    length = 0; cur = head
    while cur: length += 1; cur = cur.next
    dummy = ListNode(0, head); cur = dummy
    for _ in range(length - n): cur = cur.next
    cur.next = cur.next.next
    return dummy.next
```

Time: $O(L)$ · Space: $O(1)$

Remove nth from start. Trivial — walk and unlink.

```
def remove_nth_from_start(head, n):
    dummy = ListNode(0, head); cur = dummy
    for _ in range(n - 1): cur = cur.next
    cur.next = cur.next.next
    return dummy.next
```

Time: $O(n)$ · Space: $O(1)$

31. Reorder List — LC 143 · Medium

Problem

Reorder $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ into $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$ in place.

Optimal — three steps: middle / reverse / interleave

```
def reorder_list(head):
    if not head or not head.next: return
    slow = fast = head
    while fast.next and fast.next.next:
        slow = slow.next; fast = fast.next.next
    second = slow.next; slow.next = None
    prev = None
    while second:
        nxt = second.next; second.next = prev; prev = second; second = nxt
    first, second = head, prev
    while second:
        t1, t2 = first.next, second.next
        first.next = second; second.next = t1
        first, second = t1, t2
```

Time: $O(n)$ · Space: $O(1)$

Variations

Stack version (simpler, more memory).

```
def reorder_list_stack(head):
    if not head: return
    stack = []; cur = head
    while cur: stack.append(cur); cur = cur.next
    n = len(stack)
    cur = head
    for _ in range(n // 2):
        nxt = cur.next
        last = stack.pop()
        cur.next = last; last.next = nxt
        cur = nxt
    cur.next = None
```

Time: $O(n)$ · Space: $O(n)$

Part VII — Trees

Standard TreeNode: `python class TreeNode: def __init__(self, val=0, left=None, right=None): self.val, self.left, self.right = val, left, right`

32. Maximum Depth of Binary Tree — LC 104 · Easy

Optimal — recursion

```
def max_depth(root):
    if not root: return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

Time: $O(n)$ · Space: $O(h)$

Variations

Iterative BFS.

```
from collections import deque
def max_depth_bfs(root):
    if not root: return 0
    q = deque([root]); depth = 0
    while q:
        depth += 1
        for _ in range(len(q)):
            n = q.popleft()
            if n.left: q.append(n.left)
            if n.right: q.append(n.right)
    return depth
```

Time: $O(n)$ · Space: $O(n)$

LC 111 — Min depth. Only count leaves.

```
def min_depth(root):
    if not root: return 0
    if not root.left: return 1 + min_depth(root.right)
    if not root.right: return 1 + min_depth(root.left)
    return 1 + min(min_depth(root.left), min_depth(root.right))
```

Time: $O(n)$ · Space: $O(h)$

33. Same Tree — LC 100 · Easy

Optimal — parallel recursion

```
def is_same_tree(p, q):  
    if not p and not q: return True  
    if not p or not q: return False  
    if p.val != q.val: return False  
    return is_same_tree(p.left, q.left) and is_same_tree(p.right, q.right)
```

Time: $O(n)$ · Space: $O(h)$

Variations

LC 101 — Symmetric tree. Compare left subtree to mirror of right.

```
def is_symmetric(root):  
    def mirror(a, b):  
        if not a and not b: return True  
        if not a or not b: return False  
        return a.val == b.val and mirror(a.left, b.right) and mirror(a.right, b.left)  
    return not root or mirror(root.left, root.right)
```

Time: $O(n)$ · Space: $O(h)$

34. Invert Binary Tree — LC 226 · Easy

Optimal

```
def invert_tree(root):  
    if not root: return None  
    root.left, root.right = invert_tree(root.right), invert_tree(root.left)  
    return root
```

Time: $O(n)$ · Space: $O(h)$

Variations

Iterative BFS.

```

from collections import deque
def invert_tree_bfs(root):
    if not root: return None
    q = deque([root])
    while q:
        n = q.popleft()
        n.left, n.right = n.right, n.left
        if n.left: q.append(n.left)
        if n.right: q.append(n.right)
    return root

```

Time: O(n) · Space: O(n)

35. Binary Tree Maximum Path Sum — LC 124 · Hard

Optimal — return-down + best-passing-through

```

def max_path_sum(root):
    best = float('-inf')
    def gain(node):
        nonlocal best
        if not node: return 0
        l = max(0, gain(node.left))
        r = max(0, gain(node.right))
        best = max(best, node.val + l + r)
        return node.val + max(l, r)
    gain(root)
    return best

```

Time: O(n) · Space: O(h)

Variations

LC 543 — Diameter of binary tree. Same recursion structure with edges instead of values.

```

def diameter_of_binary_tree(root):
    best = 0
    def depth(n):
        nonlocal best
        if not n: return 0
        l = depth(n.left); r = depth(n.right)
        best = max(best, l + r)
        return 1 + max(l, r)
    depth(root)
    return best

```

Time: O(n) · Space: O(h)

LC 687 — Longest univalue path.

```
def longest_univalue_path(root):
    best = 0
    def go(n):
        nonlocal best
        if not n: return 0
        l = go(n.left); r = go(n.right)
        ll = l + 1 if n.left and n.left.val == n.val else 0
        rr = r + 1 if n.right and n.right.val == n.val else 0
        best = max(best, ll + rr)
        return max(ll, rr)
    go(root)
    return best
```

Time: $O(n)$ · Space: $O(h)$

36. Binary Tree Level Order Traversal — LC 102 · Medium

Optimal — BFS by level

```
from collections import deque
def level_order(root):
    if not root: return []
    out = []; q = deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        out.append(level)
    return out
```

Time: $O(n)$ · Space: $O(n)$

Variations

LC 107 — Bottom-up level order. Reverse the result.

```
def level_order_bottom(root):
    return level_order(root)[::-1]
```

Time: $O(n)$ · Space: $O(n)$

LC 199 — Right side view.

```
def right_side_view(root):
    if not root: return []
    out = []; q = deque([root])
    while q:
        n = len(q)
        for i in range(n):
            node = q.popleft()
            if i == n - 1: out.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
    return out
```

Time: $O(n)$ · Space: $O(n)$

LC 103 — Zigzag level order. Toggle direction per level.

```
def zigzag_level_order(root):
    if not root: return []
    out = []; q = deque([root]); ltr = True
    while q:
        level = deque()
        for _ in range(len(q)):
            node = q.popleft()
            (level.append if ltr else level.appendleft)(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        out.append(list(level)); ltr = not ltr
    return out
```

Time: $O(n)$ · Space: $O(n)$

37. Serialize and Deserialize Binary Tree — LC 297 · Hard

Optimal — preorder with null markers

```
def serialize(root):
    out = []
    def go(n):
        if not n: out.append('#'); return
        out.append(str(n.val)); go(n.left); go(n.right)
    go(root); return ','.join(out)

def deserialize(s):
    it = iter(s.split(','))
    def go():
        v = next(it)
        if v == '#': return None
        node = TreeNode(int(v))
        node.left = go(); node.right = go()
        return node
    return go()
```

Time: $O(n)$ · Space: $O(n)$

Variations

LC 449 — Serialize BST (no null markers needed).

```
def serialize_bst(root):
    out = []
    def go(n):
        if not n: return
        out.append(str(n.val)); go(n.left); go(n.right)
    go(root); return ' '.join(out)

def deserialize_bst(s):
    if not s: return None
    vals = list(map(int, s.split()))
    def go(lo, hi):
        nonlocal i
        if i == len(vals) or not (lo < vals[i] < hi): return None
        v = vals[i]; i += 1
        node = TreeNode(v)
        node.left = go(lo, v); node.right = go(v, hi)
        return node
    i = 0
    return go(float('-inf'), float('inf'))
```

Time: $O(n)$ · Space: $O(n)$

LC 428 — Serialize N-ary tree. Include child count.

```
def serialize_nary(root):
    out = []
    def go(n):
        if not n: return
        out.append(f"{n.val}#{len(n.children)}")
        for c in n.children: go(c)
    go(root); return ' '.join(out)
```

Time: $O(n)$ · Space: $O(n)$

38. Subtree of Another Tree — LC 572 · Easy

Brute force — at every node, full compare

```
def is_subtree(s, t):
    if not s: return False
    return is_same_tree(s, t) or is_subtree(s.left, t) or is_subtree(s.right, t)
```

Time: $O(n \cdot m)$ · Space: $O(h)$

Optimal — string-encode and substring match

```
def is_subtree_str(s, t):
    def serialize(n):
        if not n: return '#'
        return f'^{n.val},{serialize(n.left)},{serialize(n.right)}'
    return serialize(t) in serialize(s)
```

Time: $O(n + m)$ · Space: $O(n + m)$

Variations

Hash-based subtree matching.

```
def is_subtree_hash(s, t):
    seen = set()
    def encode(n):
        if not n: return '#'
        h = (n.val, encode(n.left), encode(n.right))
        seen.add(h)
        return h
    encode(s)
    target = encode(t)
    return target in seen
```

Time: $O(n + m)$ · Space: $O(n)$

39. Construct Binary Tree from Preorder and Inorder Traversal — LC 105 · Medium

Optimal — index map for inorder

```
def build_tree(preorder, inorder):
    idx = {v: i for i, v in enumerate(inorder)}
    pre_iter = iter(preorder)
    def go(l, r):
        if l > r: return None
        val = next(pre_iter)
        node = TreeNode(val)
        m = idx[val]
        node.left = go(l, m - 1)
        node.right = go(m + 1, r)
        return node
    return go(0, len(inorder) - 1)
```

Time: $O(n)$ · Space: $O(n)$

Variations

LC 106 — Build from inorder + postorder. Iterate postorder from the end; build right then left.

```
def build_tree_post(inorder, postorder):
    idx = {v: i for i, v in enumerate(inorder)}
    post_iter = reversed(postorder)
    def go(l, r):
        if l > r: return None
        val = next(post_iter)
        node = TreeNode(val); m = idx[val]
        node.right = go(m + 1, r) # right first!
        node.left = go(l, m - 1)
        return node
    return go(0, len(inorder) - 1)
```

Time: $O(n)$ · Space: $O(n)$

LC 889 — From preorder and postorder. Ambiguous in general; pick a side from the second preorder element.

```
def construct_from_pre_post(pre, post):
    idx = {v: i for i, v in enumerate(post)}
    def go(pl, pr, ql, qr):
        if pl > pr: return None
        node = TreeNode(pre[pl])
        if pl == pr: return node
        m = idx[pre[pl + 1]]
        size_left = m - ql + 1
        node.left = go(pl + 1, pl + size_left, ql, m)
        node.right = go(pl + size_left + 1, pr, m + 1, qr - 1)
        return node
    return go(0, len(pre) - 1, 0, len(post) - 1)
```

Time: $O(n)$ · Space: $O(n)$

40. Validate Binary Search Tree — LC 98 · Medium

Optimal — propagate (lo, hi) bounds

```
def is_valid_bst(root):
    def go(n, lo, hi):
        if not n: return True
        if not (lo < n.val < hi): return False
        return go(n.left, lo, n.val) and go(n.right, n.val, hi)
    return go(root, float('-inf'), float('inf'))
```

Time: $O(n)$ · Space: $O(h)$

Variations

In-order traversal — values must be strictly increasing.

```
def is_valid_bst_inorder(root):
    prev = [float('-inf')]
    def go(n):
        if not n: return True
        if not go(n.left): return False
        if n.val <= prev[0]: return False
        prev[0] = n.val
        return go(n.right)
    return go(root)
```

Time: $O(n)$ · Space: $O(h)$

LC 99 — Recover BST (two nodes swapped). In-order to find the two anomalies.

```
def recover_tree(root):
    first = second = prev = None
    def go(n):
        nonlocal first, second, prev
        if not n: return
        go(n.left)
        if prev and prev.val > n.val:
            if not first: first = prev
            second = n
        prev = n
        go(n.right)
    go(root)
    first.val, second.val = second.val, first.val
```

Time: $O(n)$ · Space: $O(h)$

41. Kth Smallest Element in a BST — LC 230 · Medium

Optimal — in-order traversal with early stop

```
def kth_smallest(root, k):
    stack = []; cur = root
    while cur or stack:
        while cur: stack.append(cur); cur = cur.left
        cur = stack.pop()
        k -= 1
        if k == 0: return cur.val
        cur = cur.right
```

Time: $O(h + k)$ · Space: $O(h)$

Variations

Augmented tree (with subtree sizes for repeated queries).

```

class CountedNode:
    def __init__(self, val):
        self.val = val; self.left = self.right = None; self.size = 1

def kth_smallest_augmented(root, k):
    cur = root
    while cur:
        left_size = cur.left.size if cur.left else 0
        if k == left_size + 1: return cur.val
        if k <= left_size:
            cur = cur.left
        else:
            k -= left_size + 1
            cur = cur.right

```

Time: $O(h)$ per query · Space: $O(1)$

LC 671 — Second minimum in special BT. DFS keeping track of two smallest.

```

def find_second_minimum(root):
    first = root.val
    second = float('inf')
    def go(n):
        nonlocal second
        if not n: return
        if first < n.val < second: second = n.val
        elif n.val == first:
            go(n.left); go(n.right)
    go(root)
    return -1 if second == float('inf') else second

```

Time: $O(n)$ · Space: $O(h)$

42. Lowest Common Ancestor of BST — LC 235 · Easy

Optimal — walk down using BST property

```

def lca_bst(root, p, q):
    while root:
        if p.val < root.val and q.val < root.val: root = root.left
        elif p.val > root.val and q.val > root.val: root = root.right
        else: return root

```

Time: $O(h)$ · Space: $O(1)$

Variations

LC 236 — General binary tree. Recursive — return the node if it matches p or q.

```
def lca_general(root, p, q):
    if not root or root is p or root is q: return root
    l = lca_general(root.left, p, q)
    r = lca_general(root.right, p, q)
    if l and r: return root
    return l or r
```

Time: $O(n)$ · Space: $O(h)$

LCA with parent pointers. Hash set of ancestors of p, walk q's ancestors until match.

```
def lca_with_parent(p, q):
    ancestors = set()
    while p:
        ancestors.add(p); p = p.parent
    while q:
        if q in ancestors: return q
        q = q.parent
```

Time: $O(h)$ · Space: $O(h)$

Part VIII — Tries

43. Implement Trie — LC 208 · Medium

Why

Prefix lookups in $O(L)$ regardless of dictionary size.

Code

```
class Trie:
    def __init__(self):
        self.root = {}
    def insert(self, word):
        node = self.root
        for c in word: node = node.setdefault(c, {})
        node['$'] = True
    def search(self, word):
        node = self._walk(word)
        return node is not None and node.get('$', False)
    def starts_with(self, prefix):
        return self._walk(prefix) is not None
    def _walk(self, s):
        node = self.root
        for c in s:
            if c not in node: return None
            node = node[c]
        return node
```

Time per op: $O(L)$ · Space: $O(\text{total chars})$

Variations

Trie with deletion.

```
class TrieDel(Trie):
    def delete(self, word):
        def go(node, i):
            if i == len(word):
                if not node.get('$'): return False
                node.pop('$', None)
                return len(node) == 0
            c = word[i]
            if c not in node: return False
            shrink = go(node[c], i + 1)
            if shrink:
                del node[c]
                return len(node) == 0
            return False
        go(self.root, 0)
```

Time: $O(L)$ · Space: $O(L)$ recursion

Compressed (radix) trie. Collapse single-child chains; smaller memory; insertions slightly trickier (split nodes on conflict). Useful for many similar prefixes.

44. Add and Search Word — LC 211 · Medium

Problem

Same as LC 208 but `search` may contain `'.'` matching any single character.

Optimal — Trie + DFS on dots

```
class WordDictionary:
    def __init__(self): self.root = {}
    def addWord(self, word):
        node = self.root
        for c in word: node = node.setdefault(c, {})
        node['$'] = True
    def search(self, word):
        def dfs(i, node):
            if i == len(word): return node.get('$', False)
            c = word[i]
            if c == '.':
                return any(dfs(i + 1, child) for k, child in node.items() if k != '$')
            return c in node and dfs(i + 1, node[c])
        return dfs(0, self.root)
```

Time: $O(26^L)$ worst-case (all dots); $O(L)$ typical · Space: $O(\text{total chars})$

Variations

Wildcard matching with * (any sequence). Closer to regex; can model as DFS with i, j state and memoize.

```
def is_match(word, pattern):
    @cache
    def go(i, j):
        if j == len(pattern): return i == len(word)
        first = i < len(word) and pattern[j] in (word[i], '.')
        if j + 1 < len(pattern) and pattern[j + 1] == '*':
            return go(i, j + 2) or (first and go(i + 1, j))
        return first and go(i + 1, j + 1)
    from functools import cache
    return go(0, 0)
```

Time: $O(n \cdot m)$ · *Space:* $O(n \cdot m)$

45. Word Search II — LC 212 · Hard

Problem

Given a board and a list of words, return all words that can be formed by adjacency walks (no cell reuse per word).

Brute force

For each word, run LC 79 DFS. *Time:* $O(W \cdot M \cdot N \cdot 4^L)$ · *Space:* $O(W \cdot L)$

Optimal — Trie + DFS

```
def find_words(board, words):
    root = {}
    for w in words:
        node = root
        for c in w: node = node.setdefault(c, {})
        node['$'] = w
    found = set()
    R, C = len(board), len(board[0])
    def dfs(r, c, node):
        ch = board[r][c]
        nxt = node.get(ch)
        if not nxt: return
        if '$' in nxt:
            found.add(nxt['$']); del nxt['$']
        board[r][c] = '#'
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and board[nr][nc] != '#':
                dfs(nr, nc, nxt)
        board[r][c] = ch
    for r in range(R):
        for c in range(C):
            dfs(r, c, root)
    return list(found)
```

Time: $O(M \cdot N \cdot 4^{L_{\max}})$ · *Space:* $O(\text{total chars})$

Variations

Trie pruning when leaf becomes empty. After removing a word, also clean up dead branches to speed subsequent searches.

```
def find_words_pruned(board, words):
    root = {}
    for w in words:
        node = root
        for c in w: node = node.setdefault(c, {})
        node['$'] = w
    R, C = len(board), len(board[0])
    found = []
    def dfs(r, c, parent):
        ch = board[r][c]
        node = parent.get(ch)
        if not node: return
        if '$' in node:
            found.append(node.pop('$'))
        board[r][c] = '#'
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r+dr, c+dc
            if 0 <= nr < R and 0 <= nc < C and board[nr][nc] != '#':
                dfs(nr, nc, node)
        board[r][c] = ch
        if not node: del parent[ch]
    for r in range(R):
        for c in range(C): dfs(r, c, root)
    return found
```

Time: $O(M \cdot N \cdot 4^{L_{\max}})$ · *Space:* $O(\text{total chars})$

Part IX — Heap / Priority Queue

46. Top K Frequent Elements — LC 347 · Medium

Brute force

Sort by frequency. *Time:* $O(n \log n)$ · *Space:* $O(n)$

Optimal A — heap

```
from collections import Counter
import heapq

def top_k_frequent(nums, k):
    cnt = Counter(nums)
    return [x for x, _ in heapq.nlargest(k, cnt.items(), key=lambda kv: kv[1])]
```

Time: $O(n \log k)$ · *Space:* $O(n)$

Optimal B — bucket sort

```
def top_k_frequent_bucket(nums, k):
    cnt = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)]
    for x, f in cnt.items():
        buckets[f].append(x)
    out = []
    for f in range(len(buckets) - 1, 0, -1):
        for x in buckets[f]:
            out.append(x)
            if len(out) == k: return out
    return out
```

Time: $O(n)$ · *Space:* $O(n)$

Variations

LC 692 — Top k frequent words (alphabetical tiebreak). Heap with custom key.

```
def top_k_frequent_words(words, k):
    cnt = Counter(words)
    return heapq.nsmallest(k, cnt.keys(), key=lambda w: (-cnt[w], w))
```

Time: $O(n \log k)$ · *Space:* $O(n)$

Quickselect — average $O(n)$.

```

import random
def top_k_quickselect(nums, k):
    cnt = Counter(nums)
    items = list(cnt.items())
    target = len(items) - k
    def part(l, r):
        pivot_freq = items[r][1]
        store = l
        for i in range(l, r):
            if items[i][1] < pivot_freq:
                items[i], items[store] = items[store], items[i]
                store += 1
        items[store], items[r] = items[r], items[store]
        return store
    l, r = 0, len(items) - 1
    while l < r:
        p = random.randint(l, r)
        items[p], items[r] = items[r], items[p]
        idx = part(l, r)
        if idx == target: break
        if idx < target: l = idx + 1
        else: r = idx - 1
    return [x for x, _ in items[target:]]

```

Time: $O(n)$ average · Space: $O(n)$

47. Find Median from Data Stream — LC 295 · Hard

Problem

Support `addNum(x)` and `findMedian()` over a stream.

Brute force

Sorted list. *addNum*: $O(n)$ · *findMedian*: $O(1)$

Optimal — two heaps

```

import heapq

class MedianFinder:
    def __init__(self):
        self.lo = []; self.hi = []
    def addNum(self, x):
        heapq.heappush(self.lo, -x)
        heapq.heappush(self.hi, -heapq.heappop(self.lo))
        if len(self.hi) > len(self.lo):
            heapq.heappush(self.lo, -heapq.heappop(self.hi))
    def findMedian(self):
        if len(self.lo) > len(self.hi): return -self.lo[0]
        return (-self.lo[0] + self.hi[0]) / 2

```

addNum: $O(\log n)$ · *findMedian*: $O(1)$ · Space: $O(n)$

Variations

LC 480 — Sliding-window median. Two heaps + lazy deletion via map of "to remove".

```
from sortedcontainers import SortedList
def median_sliding_window(nums, k):
    sl = SortedList()
    out = []
    for i, x in enumerate(nums):
        sl.add(x)
        if len(sl) > k:
            sl.remove(nums[i - k])
        if len(sl) == k:
            if k % 2: out.append(float(sl[k // 2]))
            else: out.append((sl[k//2 - 1] + sl[k//2]) / 2)
    return out
```

Time: $O(n \log k)$ · *Space:* $O(k)$

LC 703 — K-th largest in stream. Min-heap of size k.

```
class KthLargest:
    def __init__(self, k, nums):
        self.k = k; self.h = []
        for x in nums: self.add(x)
    def add(self, x):
        heapq.heappush(self.h, x)
        if len(self.h) > self.k: heapq.heappop(self.h)
    return self.h[0]
```

add: $O(\log k)$ · *Space:* $O(k)$

48. Last Stone Weight — LC 1046 · Easy

Optimal — max-heap (negate for Python)

```
import heapq
def last_stone_weight(stones):
    h = [-s for s in stones]
    heapq.heapify(h)
    while len(h) > 1:
        a = -heapq.heappop(h); b = -heapq.heappop(h)
        if a != b: heapq.heappush(h, -(a - b))
    return -h[0] if h else 0
```

Time: $O(n \log n)$ · *Space:* $O(n)$

Variations

LC 1049 — Last stone weight II (DP — partition halves). Reduce to subset-sum closest to total/2.

```
def last_stone_weight_ii(stones):
    total = sum(stones)
    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True
    for s in stones:
        for j in range(target, s - 1, -1):
            dp[j] = dp[j] or dp[j - s]
    for j in range(target, -1, -1):
        if dp[j]: return total - 2*j
```

Time: $O(n \cdot \text{sum})$ · Space: $O(\text{sum})$

Part X — Graphs

49. Clone Graph — LC 133 · Medium

Optimal — DFS with hash map old→new

```
def clone_graph(node):
    if not node: return None
    seen = {}
    def dfs(n):
        if n in seen: return seen[n]
        copy = type(n)(n.val)
        seen[n] = copy
        copy.neighbors = [dfs(nb) for nb in n.neighbors]
        return copy
    return dfs(node)
```

Time: $O(V + E)$ · Space: $O(V)$

Variations

BFS clone.

```
from collections import deque
def clone_graph_bfs(node):
    if not node: return None
    seen = {node: type(node)(node.val)}
    q = deque([node])
    while q:
        cur = q.popleft()
        for nb in cur.neighbors:
            if nb not in seen:
                seen[nb] = type(nb)(nb.val)
                q.append(nb)
            seen[cur].neighbors.append(seen[nb])
    return seen[node]
```

Time: $O(V + E)$ · Space: $O(V)$

LC 138 — Copy list with random pointer.

```
def copy_random_list(head):
    if not head: return None
    seen = {}
    def go(n):
        if not n: return None
        if n in seen: return seen[n]
        copy = type(n)(n.val)
        seen[n] = copy
        copy.next = go(n.next)
        copy.random = go(n.random)
        return copy
    return go(head)
```

Time: $O(n)$ · Space: $O(n)$

50. Course Schedule — LC 207 · Medium

Optimal — BFS (Kahn's algorithm)

```
from collections import deque, defaultdict
def can_finish(numCourses, prerequisites):
    indeg = [0] * numCourses
    g = defaultdict(list)
    for c, pre in prerequisites:
        g[pre].append(c); indeg[c] += 1
    q = deque(i for i, d in enumerate(indeg) if d == 0)
    done = 0
    while q:
        u = q.popleft(); done += 1
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0: q.append(v)
    return done == numCourses
```

Time: $O(V + E)$ · Space: $O(V + E)$

Variations

LC 210 — Find an order. Capture the BFS order itself.

```
def find_order(numCourses, prerequisites):
    indeg = [0] * numCourses
    g = defaultdict(list)
    for c, pre in prerequisites:
        g[pre].append(c); indeg[c] += 1
    q = deque(i for i, d in enumerate(indeg) if d == 0)
    out = []
    while q:
        u = q.popleft(); out.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0: q.append(v)
    return out if len(out) == numCourses else []
```

Time: $O(V + E)$ · *Space:* $O(V + E)$

DFS with color states (cycle detection).

```
WHITE, GRAY, BLACK = 0, 1, 2
def can_finish_dfs(numCourses, prerequisites):
    g = defaultdict(list)
    for c, pre in prerequisites: g[pre].append(c)
    color = [WHITE] * numCourses
    def dfs(u):
        if color[u] == GRAY: return False
        if color[u] == BLACK: return True
        color[u] = GRAY
        for v in g[u]:
            if not dfs(v): return False
        color[u] = BLACK
        return True
    return all(dfs(i) for i in range(numCourses))
```

Time: $O(V + E)$ · *Space:* $O(V + E)$

51. Pacific Atlantic Water Flow — LC 417 · Medium

Brute force

DFS from each cell. *Time:* $O((MN)^2)$ · *Space:* $O(MN)$

Optimal — flip the flow: BFS/DFS from each ocean

```
def pacific_atlantic(heights):
    if not heights: return []
    R, C = len(heights), len(heights[0])
    pac = set(); atl = set()
    def dfs(r, c, seen):
        seen.add((r, c))
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r+dr, c+dc
            if 0 <= nr < R and 0 <= nc < C and (nr, nc) not in seen and heights[nr][nc] >= heights[r][c]:
                dfs(nr, nc, seen)
    for c in range(C):
        dfs(0, c, pac); dfs(R-1, c, atl)
    for r in range(R):
        dfs(r, 0, pac); dfs(r, C-1, atl)
    return [list(rc) for rc in pac & atl]
```

Time: $O(MN)$ · Space: $O(MN)$

Variations

Single BFS from a specific ocean (utility).

```
def reachable_from_borders(heights, borders):
    R, C = len(heights), len(heights[0])
    seen = set(borders)
    q = deque(borders)
    while q:
        r, c = q.popleft()
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r+dr, c+dc
            if 0 <= nr < R and 0 <= nc < C and (nr, nc) not in seen and heights[nr][nc] >= heights[r][c]:
                seen.add((nr, nc)); q.append((nr, nc))
    return seen
```

Time: $O(MN)$ · Space: $O(MN)$

52. Number of Islands — LC 200 · Medium

Optimal — DFS sink

```
def num_islands(grid):
    if not grid: return 0
    R, C = len(grid), len(grid[0])
    def sink(r, c):
        if r < 0 or r >= R or c < 0 or c >= C or grid[r][c] != '1': return
        grid[r][c] = '#'
        sink(r+1,c); sink(r-1,c); sink(r,c+1); sink(r,c-1)
    count = 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] == '1': sink(r, c); count += 1
    return count
```

Time: $O(MN)$ · Space: $O(MN)$ recursion

Variations

LC 695 — Max area of island. Same DFS, return area.

```
def max_area_of_island(grid):
    R, C = len(grid), len(grid[0])
    def area(r, c):
        if r < 0 or r >= R or c < 0 or c >= C or grid[r][c] != 1: return 0
        grid[r][c] = 0
        return 1 + area(r+1,c) + area(r-1,c) + area(r,c+1) + area(r,c-1)
    return max((area(r, c) for r in range(R) for c in range(C) if grid[r][c] == 1), default=0)
```

Time: $O(MN)$ · Space: $O(MN)$

LC 130 — Surrounded regions. Flood-fill from borders to mark "safe", then capture the rest. (Also see U10.)

LC 305 — Number of islands II (online). Union-Find on each insertion. (See U11.)

LC 1254 — Number of closed islands. Like LC 200 but skip islands that touch the border.

```
def closed_island(grid):
    R, C = len(grid), len(grid[0])
    def fill(r, c):
        if r < 0 or r >= R or c < 0 or c >= C: return False
        if grid[r][c] != 0: return True
        grid[r][c] = 1
        d = fill(r+1,c); u = fill(r-1,c); l = fill(r,c+1); ri = fill(r,c-1)
        return d and u and l and ri
    return sum(1 for r in range(1, R-1) for c in range(1, C-1) if grid[r][c] == 0 and fill(r, c))
```

Time: $O(MN)$ · Space: $O(MN)$

53. Longest Consecutive Sequence — LC 128 · Medium

Brute force

Sort, sweep. *Time:* $O(n \log n)$ · *Space:* $O(n)$

Optimal — set, only start from sequence "starts"

```
def longest_consecutive(nums):
    s = set(nums)
    best = 0
    for x in s:
        if x - 1 not in s:
            cur = x; length = 1
            while cur + 1 in s:
                cur += 1; length += 1
            best = max(best, length)
    return best
```

Time: $O(n)$ · *Space:* $O(n)$

Variations

Union-Find based.

```
def longest_consecutive_uf(nums):
    parent = {x: x for x in nums}
    size = {x: 1 for x in nums}
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]; x = parent[x]
        return x
    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb: return
        if size[ra] < size[rb]: ra, rb = rb, ra
        parent[rb] = ra; size[ra] += size[rb]
    for x in nums:
        if x + 1 in parent: union(x, x + 1)
    return max(size.values(), default=0)
```

Time: $O(n \alpha(n))$ · *Space:* $O(n)$

54. Word Search — LC 79 · Medium

Optimal — DFS with in-place marking

```
def exist(board, word):
    R, C = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word): return True
        if r < 0 or r >= R or c < 0 or c >= C or board[r][c] != word[i]: return False
        ch = board[r][c]; board[r][c] = '#'
        ok = (dfs(r+1, c, i+1) or dfs(r-1, c, i+1) or dfs(r, c+1, i+1) or dfs(r, c-1, i+1))
        board[r][c] = ch
        return ok
    for r in range(R):
        for c in range(C):
            if dfs(r, c, 0): return True
    return False
```

Time: $O(MN \cdot 4^L)$ · Space: $O(L)$ recursion

Variations

LC 212 — Word Search II (multiple words). See problem 45.

With diagonal moves (8-direction). Add 4 more dr/dc in the loop.

```
def exist_diag(board, word):
    R, C = len(board), len(board[0])
    DIRS = [(dr, dc) for dr in (-1, 0, 1) for dc in (-1, 0, 1) if (dr or dc)]
    def dfs(r, c, i):
        if i == len(word): return True
        if r < 0 or r >= R or c < 0 or c >= C or board[r][c] != word[i]: return False
        ch = board[r][c]; board[r][c] = '#'
        ok = any(dfs(r+dr, c+dc, i+1) for dr, dc in DIRS)
        board[r][c] = ch
        return ok
    return any(dfs(r, c, 0) for r in range(R) for c in range(C))
```

Time: $O(MN \cdot 8^L)$ · Space: $O(L)$

55. Number of Connected Components — LC 323 · Medium (LC Premium)

Optimal — Union Find

```
class DSU:
    def __init__(self, n):
        self.p = list(range(n)); self.r = [0]*n
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]
            x = self.p[x]
        return x
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb: return False
        if self.r[ra] < self.r[rb]: ra, rb = rb, ra
        self.p[rb] = ra
        if self.r[ra] == self.r[rb]: self.r[ra] += 1
        return True

def count_components(n, edges):
    dsu = DSU(n); components = n
    for a, b in edges:
        if dsu.union(a, b): components -= 1
    return components
```

Time: $O((V + E) \alpha(V))$ · Space: $O(V)$

Variations

DFS / BFS variant.

```
def count_components_dfs(n, edges):
    g = defaultdict(list)
    for a, b in edges:
        g[a].append(b); g[b].append(a)
    seen = set(); comps = 0
    for s in range(n):
        if s in seen: continue
        comps += 1
        stack = [s]
        while stack:
            u = stack.pop()
            if u in seen: continue
            seen.add(u)
            stack.extend(g[u])
    return comps
```

Time: $O(V + E)$ · Space: $O(V + E)$

56. Graph Valid Tree — LC 261 · Medium (LC Premium)

Definition

An undirected graph is a tree iff $n - 1$ edges and connected.

Optimal — DSU

```
def valid_tree(n, edges):
    if len(edges) != n - 1: return False
    dsu = DSU(n)
    for a, b in edges:
        if not dsu.union(a, b): return False
    return True
```

Time: $O((V + E) \alpha(V))$ · Space: $O(V)$

Variations

DFS-based — detect cycle and check reachability.

```
def valid_tree_dfs(n, edges):
    if len(edges) != n - 1: return False
    g = defaultdict(list)
    for a, b in edges: g[a].append(b); g[b].append(a)
    seen = set()
    def dfs(u, parent):
        if u in seen: return False
        seen.add(u)
        for v in g[u]:
            if v == parent: continue
            if not dfs(v, u): return False
        return True
    return dfs(0, -1) and len(seen) == n
```

Time: $O(V + E)$ · Space: $O(V + E)$

Part XI — Dynamic Programming (1D)

57. Climbing Stairs — LC 70 · Easy

Brute force

Recursion. Time: $O(2^n)$ · Space: $O(n)$

Optimal — DP with two variables

```
def climb_stairs(n):
    a, b = 1, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Time: $O(n)$ · Space: $O(1)$

Variations

Steps in {1, 2, 3}.

```
def climb_stairs_three(n):
    if n == 0: return 1
    if n == 1: return 1
    if n == 2: return 2
    a, b, c = 1, 1, 2
    for _ in range(n - 2):
        a, b, c = b, c, a + b + c
    return c
```

Time: $O(n)$ · Space: $O(1)$

LC 746 — Min cost climbing stairs.

```
def min_cost_climbing_stairs(cost):
    a = b = 0
    for x in cost:
        a, b = b, min(a, b) + x
    return min(a, b)
```

Time: $O(n)$ · Space: $O(1)$

Steps in arbitrary set.

```
def climb_stairs_set(n, steps):
    dp = [0] * (n + 1)
    dp[0] = 1
    for i in range(1, n + 1):
        for s in steps:
            if i >= s: dp[i] += dp[i - s]
    return dp[n]
```

Time: $O(n \cdot |\text{steps}|)$ · Space: $O(n)$

58. Coin Change — LC 322 · Medium

Brute force

Recursion. Exponential.

Optimal — bottom-up DP

```
def coin_change(coins, amount):
    INF = amount + 1
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a: dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1
```

Time: $O(t \cdot |\text{coins}|)$ · Space: $O(t)$

Variations

LC 518 — Number of ways to make amount. Outer loop over coins (key for unique ways!).

```
def change(amount, coins):
    dp = [0] * (amount + 1); dp[0] = 1
    for c in coins:
        for a in range(c, amount + 1):
            dp[a] += dp[a - c]
    return dp[amount]
```

Time: $O(t \cdot |\text{coins}|)$ · Space: $O(t)$

Bounded coin change (each coin $\leq k_i$ times).

```
def coin_change_bounded(coins, counts, amount):
    INF = amount + 1
    dp = [0] + [INF] * amount
    for c, k in zip(coins, counts):
        # binary lifting decomposition
        copies = []
        j = 1
        while j <= k:
            copies.append(j); k -= j; j *= 2
        if k > 0: copies.append(k)
        for cnt in copies:
            v = c * cnt
            for a in range(amount, v - 1, -1):
                dp[a] = min(dp[a], dp[a - v] + cnt)
    return dp[amount] if dp[amount] != INF else -1
```

Time: $O(\text{amount} \cdot \sum \log k_i)$ · Space: $O(\text{amount})$

59. Longest Increasing Subsequence — LC 300 · Medium

Brute force / DP

```
def length_of_lis_dp(nums):
    if not nums: return 0
    dp = [1]*len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]: dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)
```

Time: $O(n^2)$ · Space: $O(n)$

Optimal — patience sorting with binary search

```
from bisect import bisect_left
def length_of_lis(nums):
    tails = []
    for x in nums:
        i = bisect_left(tails, x)
        if i == len(tails): tails.append(x)
        else: tails[i] = x
    return len(tails)
```

Time: $O(n \log n)$ · Space: $O(n)$

Variations

Reconstruct the actual sequence.

```
def lis_sequence(nums):
    if not nums: return []
    tails = []; tails_idx = []; prev = [-1]*len(nums)
    for i, x in enumerate(nums):
        j = bisect_left(tails, x)
        if j == len(tails):
            tails.append(x); tails_idx.append(i)
        else:
            tails[j] = x; tails_idx[j] = i
        prev[i] = tails_idx[j-1] if j > 0 else -1
    out = []; k = tails_idx[-1]
    while k != -1:
        out.append(nums[k]); k = prev[k]
    return out[::-1]
```

Time: $O(n \log n)$ · Space: $O(n)$

LC 354 — Russian doll envelopes. Sort by w asc, h desc, then LIS on h.

```
def max_envelopes(envelopes):
    envelopes.sort(key=lambda e: (e[0], -e[1]))
    return length_of_lis([h for _, h in envelopes])
```

Time: $O(n \log n)$ · Space: $O(n)$

LC 673 — Number of LIS. DP carrying counts.

```
def find_number_of_lis(nums):
    n = len(nums)
    length = [1]*n; count = [1]*n
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i]:
                if length[j] + 1 > length[i]:
                    length[i] = length[j] + 1; count[i] = count[j]
                elif length[j] + 1 == length[i]:
                    count[i] += count[j]
    longest = max(length)
    return sum(c for l, c in zip(length, count) if l == longest)
```

Time: $O(n^2)$ · Space: $O(n)$

60. Word Break — LC 139 · Medium

Brute force

Recursive. Exponential.

Optimal — DP

```
def word_break(s, wordDict):
    words = set(wordDict)
    n = len(s)
    dp = [False]*(n+1); dp[0] = True
    for i in range(1, n+1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True; break
    return dp[n]
```

Time: $O(n^2)$ · Space: $O(n)$

Variations

LC 140 — Return all sentences. Memoized DFS.

```
def word_break_ii(s, wordDict):
    words = set(wordDict)
    @cache
    def go(i):
        if i == len(s): return [[]]
        out = []
        for j in range(i + 1, len(s) + 1):
            w = s[i:j]
            if w in words:
                for tail in go(j):
                    out.append([w] + tail)
        return out
    from functools import cache
    return [' '.join(parts) for parts in go(0)]
```

Time: $O(n \cdot 2^n)$ worst-case · Space: $O(2^n)$

Longest prefix of s that's a dict word.

```
def longest_prefix_in_dict(s, words):
    words = set(words)
    best = ''
    for i in range(1, len(s) + 1):
        if s[:i] in words: best = s[:i]
    return best
```

Time: $O(n^2)$ (substring) or $O(n)$ with Trie · Space: $O(n)$

61. Combination Sum — LC 39 · Medium

Optimal — backtracking with start index

```
def combination_sum(candidates, target):
    candidates.sort()
    out = []
    def back(start, remain, path):
        if remain == 0: out.append(path[:]); return
        for i in range(start, len(candidates)):
            x = candidates[i]
            if x > remain: break
            path.append(x)
            back(i, remain - x, path)
            path.pop()
    back(0, target, [])
    return out
```

Time: exponential · Space: $O(\text{target})$

Variations

LC 40 — Each used once + dedup.

```
def combination_sum_ii(candidates, target):
    candidates.sort(); out = []
    def back(start, remain, path):
        if remain == 0: out.append(path[:]); return
        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i-1]: continue
            x = candidates[i]
            if x > remain: break
            path.append(x); back(i + 1, remain - x, path); path.pop()
    back(0, target, [])
    return out
```

Time: exponential · Space: $O(\text{target})$

LC 377 — Count combinations (order matters). DP.

```
def combination_sum_iv(nums, target):
    dp = [0]*(target+1); dp[0] = 1
    for a in range(1, target+1):
        for x in nums:
            if a >= x: dp[a] += dp[a - x]
    return dp[target]
```

Time: $O(\text{target} \cdot n)$ · Space: $O(\text{target})$

LC 216 — Combinations of [1..9] summing to n with k picks.

```
def combination_sum_iii(k, n):
    out = []
    def back(start, remain, path):
        if len(path) == k and remain == 0: out.append(path[:]); return
        if len(path) >= k or remain < 0: return
        for x in range(start, 10):
            path.append(x); back(x + 1, remain - x, path); path.pop()
    back(1, n, [])
    return out
```

Time: $C(9,k)$ · Space: $O(k)$

62. House Robber — LC 198 · Medium

Optimal — DP

```
def rob(nums):
    prev2 = prev1 = 0
    for x in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + x)
    return prev1
```

Time: $O(n)$ · Space: $O(1)$

Variations

LC 213 — Circular. See problem 63.

LC 337 — Binary tree houses. Tree DP returning (rob, skip).

```
def rob_tree(root):
    def go(n):
        if not n: return (0, 0)
        l = go(n.left); r = go(n.right)
        rob_now = n.val + l[1] + r[1]
        skip_now = max(l) + max(r)
        return (rob_now, skip_now)
    return max(go(root))
```

Time: $O(n)$ · *Space:* $O(h)$

LC 740 — Delete and earn. Map to House Robber by value.

```
def delete_and_earn(nums):
    if not nums: return 0
    cnt = Counter(nums)
    mx = max(nums)
    points = [i * cnt[i] for i in range(mx + 1)]
    prev2 = prev1 = 0
    for p in points:
        prev2, prev1 = prev1, max(prev1, prev2 + p)
    return prev1
```

Time: $O(n + \max)$ · *Space:* $O(\max)$

63. House Robber II — LC 213 · Medium

Optimal — two LC 198 runs

```
def rob_circular(nums):
    if len(nums) == 1: return nums[0]
    def linear(arr):
        a = b = 0
        for x in arr:
            a, b = b, max(b, a + x)
        return b
    return max(linear(nums[:-1]), linear(nums[1:]))
```

Time: $O(n)$ · *Space:* $O(1)$

Variations

LC 656 — Minimum cost path with K jumps (DP variant). Different problem flavor; mention only.

64. Decode Ways — LC 91 · Medium

Brute force

Recurse. *Time:* $O(2^n)$

Optimal — DP with rolling vars

```
def num_decodings(s):
    if not s or s[0] == '0': return 0
    prev2, prev1 = 1, 1
    for i in range(1, len(s)):
        cur = 0
        if s[i] != '0': cur += prev1
        two = int(s[i-1:i+1])
        if 10 <= two <= 26: cur += prev2
        prev2, prev1 = prev1, cur
    return prev1
```

Time: $O(n)$ · *Space:* $O(1)$

Variations

LC 639 — Decode ways with *. Each * represents 1–9.

```
MOD = 10**9 + 7
def num_decodings_star(s):
    if not s: return 0
    n = len(s)
    dp = [0]*(n+1); dp[0] = 1
    for i in range(1, n+1):
        c = s[i-1]
        if c == '*': dp[i] = dp[i-1] * 9
        elif c != '0': dp[i] = dp[i-1]
        if i >= 2:
            two = s[i-2:i]
            if two[0] == '*' and two[1] == '*':
                dp[i] += dp[i-2] * 15
            elif two[0] == '*':
                dp[i] += dp[i-2] * (2 if two[1] <= '6' else 1)
            elif two[1] == '*':
                if two[0] == '1': dp[i] += dp[i-2] * 9
                elif two[0] == '2': dp[i] += dp[i-2] * 6
            else:
                v = int(two)
                if 10 <= v <= 26: dp[i] += dp[i-2]
        dp[i] %= MOD
    return dp[n]
```

Time: $O(n)$ · *Space:* $O(n)$

Part XII — Dynamic Programming (2D)

65. Longest Common Subsequence — LC 1143 · Medium

Optimal — 2D DP

```
def lcs(a, b):
    n, m = len(a), len(b)
    dp = [[0]*(m+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for j in range(1, m+1):
            if a[i-1] == b[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[n][m]
```

Time: $O(nm)$ · Space: $O(nm)$

Variations

Space-optimized to $O(\min(n, m))$ two rows.

```
def lcs_two_rows(a, b):
    if len(a) < len(b): a, b = b, a
    prev = [0]*(len(b)+1); cur = [0]*(len(b)+1)
    for i in range(1, len(a)+1):
        for j in range(1, len(b)+1):
            if a[i-1] == b[j-1]: cur[j] = prev[j-1] + 1
            else: cur[j] = max(prev[j], cur[j-1])
        prev, cur = cur, [0]*(len(b)+1)
    return prev[-1]
```

Time: $O(nm)$ · Space: $O(\min(n, m))$

LC 583 — Delete operations. $n + m - 2 * \text{LCS}$.

```
def min_delete_distance(a, b):
    return len(a) + len(b) - 2 * lcs(a, b)
```

Time: $O(nm)$ · Space: $O(nm)$

LC 1035 — Uncrossed lines. Literally LCS in disguise.

```
def max_uncrossed_lines(A, B):
    return lcs(A, B)
```

Time: $O(nm)$ · Space: $O(nm)$

LC 516 — Longest palindromic subsequence. LCS of `s` and reversed `s`.

```
def longest_palindrome_subseq(s):
    return lcs(s, s[::-1])
```

Time: $O(n^2)$ · Space: $O(n^2)$

66. Unique Paths — LC 62 · Medium

Optimal — DP one row

```
def unique_paths(m, n):
    row = [1] * n
    for _ in range(1, m):
        for j in range(1, n):
            row[j] += row[j - 1]
    return row[n - 1]
```

Time: $O(mn)$ · Space: $O(n)$

Variations

Math — combinatorial formula.

```
from math import comb
def unique_paths_math(m, n):
    return comb(m + n - 2, m - 1)
```

Time: $O(\min(m, n))$ · Space: $O(1)$

LC 63 — With obstacles. Zero out blocked cells.

```
def unique_paths_obstacles(grid):
    R, C = len(grid), len(grid[0])
    dp = [0] * C; dp[0] = 1 if grid[0][0] == 0 else 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] == 1: dp[c] = 0
            elif c > 0: dp[c] += dp[c-1]
    return dp[-1]
```

Time: $O(MN)$ · Space: $O(N)$

LC 980 — Visit every cell exactly once. Backtracking from start.

```

def unique_paths_iii(grid):
    R, C = len(grid), len(grid[0])
    empty = 0; sr = sc = -1
    for r in range(R):
        for c in range(C):
            if grid[r][c] == 0: empty += 1
            elif grid[r][c] == 1: sr, sc = r, c
    count = 0
    def dfs(r, c, remaining):
        nonlocal count
        if r < 0 or r >= R or c < 0 or c >= C or grid[r][c] == -1: return
        if grid[r][c] == 2:
            if remaining == 0: count += 1
            return
        grid[r][c] = -1
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            dfs(r+dr, c+dc, remaining - 1)
        grid[r][c] = 0
    dfs(sr, sc, empty + 1)
    return count

```

Time: exponential · Space: O(MN)

67. Edit Distance — LC 72 · Medium

Optimal — 2D DP

```

def min_distance(a, b):
    n, m = len(a), len(b)
    dp = [[0]*(m+1) for _ in range(n+1)]
    for i in range(n+1): dp[i][0] = i
    for j in range(m+1): dp[0][j] = j
    for i in range(1, n+1):
        for j in range(1, m+1):
            if a[i-1] == b[j-1]: dp[i][j] = dp[i-1][j-1]
            else: dp[i][j] = 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])
    return dp[n][m]

```

Time: O(nm) · Space: O(nm)

Variations

LC 161 — One edit distance. Verify in O(n).

```

def is_one_edit_distance(s, t):
    if abs(len(s) - len(t)) > 1: return False
    if len(s) > len(t): s, t = t, s
    for i in range(len(s)):
        if s[i] != t[i]:
            if len(s) == len(t):
                return s[i+1:] == t[i+1:]
            return s[i:] == t[i+1:]
    return len(s) + 1 == len(t)

```

Time: $O(n)$ · Space: $O(1)$

Space-optimized 1D DP.

```
def edit_distance_1d(a, b):
    if len(a) < len(b): a, b = b, a
    prev = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        cur = [i] + [0]*len(b)
        for j in range(1, len(b) + 1):
            if a[i-1] == b[j-1]: cur[j] = prev[j-1]
            else: cur[j] = 1 + min(prev[j-1], prev[j], cur[j-1])
        prev = cur
    return prev[-1]
```

Time: $O(nm)$ · Space: $O(\min(n, m))$

LC 583 — Only deletions allowed. See LCS variation above.

Part XIII — Greedy

68. Jump Game — LC 55 · Medium

Brute force

DP from each index. Time: $O(n^2)$ · Space: $O(n)$

Optimal — greedy reach

```
def can_jump(nums):
    reach = 0
    for i, x in enumerate(nums):
        if i > reach: return False
        reach = max(reach, i + x)
    return True
```

Time: $O(n)$ · Space: $O(1)$

Variations

LC 45 — Min jumps to last index. BFS-flavored greedy with `current_end` and `farthest`.

```
def jump(nums):
    jumps = current_end = farthest = 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == current_end:
            jumps += 1; current_end = farthest
    return jumps
```

Time: $O(n)$ · Space: $O(1)$

LC 1306 — Jump game III (boolean BFS).

```
def can_reach(arr, start):
    n = len(arr); seen = [False]*n; q = deque([start])
    while q:
        i = q.popleft()
        if not (0 <= i < n) or seen[i]: continue
        if arr[i] == 0: return True
        seen[i] = True
        q.append(i + arr[i]); q.append(i - arr[i])
    return False
```

Time: $O(n)$ · Space: $O(n)$

LC 1340 — Jump game V. DP with memo.

```
def max_jumps(arr, d):
    n = len(arr)
    @cache
    def go(i):
        best = 1
        for s in (-1, 1):
            for k in range(1, d + 1):
                j = i + s*k
                if not (0 <= j < n) or arr[j] >= arr[i]: break
                best = max(best, 1 + go(j))
        return best
    from functools import cache
    return max(go(i) for i in range(n))
```

Time: $O(n \cdot d)$ · Space: $O(n)$

Part XIV — Intervals

69. Insert Interval — LC 57 · Medium

Optimal — single pass

```
def insert(intervals, newInterval):
    out = []; i, n = 0, len(intervals)
    while i < n and intervals[i][1] < newInterval[0]:
        out.append(intervals[i]); i += 1
    while i < n and intervals[i][0] <= newInterval[1]:
        newInterval = [min(newInterval[0], intervals[i][0]),
                        max(newInterval[1], intervals[i][1])]
        i += 1
    out.append(newInterval)
    while i < n:
        out.append(intervals[i]); i += 1
    return out
```

Time: $O(n)$ · Space: $O(n)$

Variations

Insert + merge variant — already-merged input. This is the canonical version — no extra logic needed.

Online insert (many calls). Maintain sorted list; binary search the insert point; merge neighbors.

```
from bisect import insert
class IntervalSet:
    def __init__(self):
        self.starts = []
        self.ends = {}
    def insert(self, s, e):
        insert(self.starts, s)
        self.ends[s] = e
        # merge after; complex for production — use sortedcontainers or a tree map.
```

Insert: $O(n)$ worst (Python list) · Space: $O(n)$

70. Merge Intervals — LC 56 · Medium

Optimal — sort by start

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    out = []
    for s, e in intervals:
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

Time: $O(n \log n)$ · Space: $O(n)$

Variations

Merge with weights/values (event scheduling). Same merge, carry extra fields.

```
def merge_with_value(intervals):
    intervals.sort(key=lambda x: x[0])
    out = []
    for s, e, v in intervals:
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
            out[-1][2] += v
        else:
            out.append([s, e, v])
    return out
```

Time: $O(n \log n)$ · Space: $O(n)$

LC 759 — Employee free time. Merge then take gaps.

```
def employee_free_time(schedule):
    flat = sorted([iv for emp in schedule for iv in emp], key=lambda iv: iv[0])
    merged = []
    for s, e in flat:
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return [[merged[i-1][1], merged[i][0]] for i in range(1, len(merged))]
```

Time: $O(n \log n)$ · *Space:* $O(n)$

71. Non-overlapping Intervals — LC 435 · Medium

Optimal — greedy by earliest end time

```
def erase_overlap(intervals):
    intervals.sort(key=lambda x: x[1])
    end = float('-inf'); removed = 0
    for s, e in intervals:
        if s >= end: end = e
        else: removed += 1
    return removed
```

Time: $O(n \log n)$ · *Space:* $O(1)$

Variations

LC 452 — Min arrows to burst balloons. Same greedy by end.

```
def find_min_arrow_shots(points):
    points.sort(key=lambda p: p[1])
    arrows = 0; end = float('-inf')
    for s, e in points:
        if s > end:
            arrows += 1; end = e
    return arrows
```

Time: $O(n \log n)$ · *Space:* $O(1)$

72. Meeting Rooms II — LC 253 · Medium (LC Premium)

Optimal A — heap

```
import heapq
def min_meeting_rooms(intervals):
    if not intervals: return 0
    intervals.sort(key=lambda x: x[0])
    heap = []
    for s, e in intervals:
        if heap and heap[0] <= s: heapq.heappop(heap)
        heapq.heappush(heap, e)
    return len(heap)
```

Time: $O(n \log n)$ · Space: $O(n)$

Optimal B — sweep line

```
def min_meeting_rooms_sweep(intervals):
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = used = j = 0
    for s in starts:
        while j < len(ends) and ends[j] <= s: j += 1; used -= 1
        used += 1; rooms = max(rooms, used)
    return rooms
```

Time: $O(n \log n)$ · Space: $O(n)$

Variations

LC 252 — Can attend all meetings? Sort by start; check no overlaps.

```
def can_attend(intervals):
    intervals.sort(key=lambda x: x[0])
    return all(intervals[i][0] >= intervals[i-1][1] for i in range(1, len(intervals)))
```

Time: $O(n \log n)$ · Space: $O(1)$

LC 729 — My Calendar I. Each booking must not overlap.

```
class MyCalendar:
    def __init__(self): self.bookings = []
    def book(self, s, e):
        for bs, be in self.bookings:
            if s < be and bs < e: return False
        self.bookings.append((s, e))
        return True
```

book: $O(n)$ per call · Space: $O(n)$

LC 731 — My Calendar II (double bookings allowed, no triple).

```

class MyCalendarTwo:
    def __init__(self):
        self.book = [] # all bookings
        self.overlaps = [] # double bookings
    def book(self, s, e):
        for os, oe in self.overlaps:
            if s < oe and os < e: return False
        for bs, be in self.book:
            if s < be and bs < e:
                self.overlaps.append((max(s, bs), min(e, be)))
        self.book.append((s, e))
        return True

```

book: O(n) · Space: O(n)

LC 732 — My Calendar III (max k overlap). Sweep with sorted map.

```

from sortedcontainers import SortedDict
class MyCalendarThree:
    def __init__(self): self.delta = SortedDict()
    def book(self, s, e):
        self.delta[s] = self.delta.get(s, 0) + 1
        self.delta[e] = self.delta.get(e, 0) - 1
        active = best = 0
        for v in self.delta.values():
            active += v; best = max(best, active)
        return best

```

book: O(n) · Space: O(n)

Part XV — Matrix

73. Set Matrix Zeroes — LC 73 · Medium

Brute force

Track sets of zero rows/cols. *Time: O(MN) · Space: O(M+N)*

Optimal — encode flags into the first row and column

```
def set_zeroes(m):
    R, C = len(m), len(m[0])
    first_row = any(m[0][c] == 0 for c in range(C))
    first_col = any(m[r][0] == 0 for r in range(R))
    for r in range(1, R):
        for c in range(1, C):
            if m[r][c] == 0:
                m[r][0] = 0; m[0][c] = 0
    for r in range(1, R):
        for c in range(1, C):
            if m[r][0] == 0 or m[0][c] == 0: m[r][c] = 0
    if first_row:
        for c in range(C): m[0][c] = 0
    if first_col:
        for r in range(R): m[r][0] = 0
```

Time: $O(MN)$ · *Space:* $O(1)$

Variations

With sets (clearer code, slightly more memory).

```
def set_zeroes_sets(m):
    R, C = len(m), len(m[0])
    rows, cols = set(), set()
    for r in range(R):
        for c in range(C):
            if m[r][c] == 0: rows.add(r); cols.add(c)
    for r in range(R):
        for c in range(C):
            if r in rows or c in cols: m[r][c] = 0
```

Time: $O(MN)$ · *Space:* $O(M+N)$

74. Spiral Matrix — LC 54 · Medium

Optimal — four boundaries

```
def spiral_order(m):
    out = []
    if not m: return out
    top, bot = 0, len(m) - 1
    left, right = 0, len(m[0]) - 1
    while top <= bot and left <= right:
        for c in range(left, right + 1): out.append(m[top][c])
        top += 1
        for r in range(top, bot + 1): out.append(m[r][right])
        right -= 1
        if top <= bot:
            for c in range(right, left - 1, -1): out.append(m[bot][c])
            bot -= 1
        if left <= right:
            for r in range(bot, top - 1, -1): out.append(m[r][left])
            left += 1
    return out
```

Time: $O(MN)$ · Space: $O(1)$ extra

Variations

LC 59 — Generate spiral matrix. Same boundary loop, fill values $1..n^2$.

```
def generate_matrix(n):
    m = [[0]*n for _ in range(n)]
    top, bot, left, right = 0, n-1, 0, n-1
    v = 1
    while v <= n*n:
        for c in range(left, right+1): m[top][c] = v; v += 1
        top += 1
        for r in range(top, bot+1): m[r][right] = v; v += 1
        right -= 1
        for c in range(right, left-1, -1): m[bot][c] = v; v += 1
        bot -= 1
        for r in range(bot, top-1, -1): m[r][left] = v; v += 1
        left += 1
    return m
```

Time: $O(n^2)$ · Space: $O(n^2)$

LC 885 — Spiral matrix III (start anywhere, bounded).

```
def spiral_matrix_iii(R, C, r, c):
    out = [[r, c]]; dr = [0,1,0,-1]; dc = [1,0,-1,0]
    step = 1; d = 0
    while len(out) < R * C:
        for _ in range(2):
            for _ in range(step):
                r += dr[d]; c += dc[d]
                if 0 <= r < R and 0 <= c < C: out.append([r, c])
            d = (d + 1) % 4
        step += 1
    return out
```

Time: $O(\max(R,C)^2)$ · Space: $O(RC)$

75. Rotate Image — LC 48 · Medium

Optimal — transpose then reverse each row

```
def rotate(m):
    n = len(m)
    for i in range(n):
        for j in range(i + 1, n):
            m[i][j], m[j][i] = m[j][i], m[i][j]
    for row in m:
        row.reverse()
```

Time: $O(n^2)$ · Space: $O(1)$

Variations

Counter-clockwise rotation. Transpose then reverse columns (i.e. flip vertically), or reverse rows then transpose.

```
def rotate_ccw(m):
    n = len(m)
    for i in range(n):
        for j in range(i + 1, n):
            m[i][j], m[j][i] = m[j][i], m[i][j]
    m.reverse()
```

Time: $O(n^2)$ · Space: $O(1)$

180° rotation. Reverse rows then reverse each row.

```
def rotate_180(m):
    m.reverse()
    for row in m:
        row.reverse()
```

Time: $O(n^2)$ · Space: $O(1)$

Layer-by-layer rotation (no transpose). Older interview style; same complexity.

```
def rotate_layer(m):
    n = len(m)
    for layer in range(n // 2):
        first, last = layer, n - 1 - layer
        for i in range(first, last):
            offset = i - first
            top = m[first][i]
            m[first][i] = m[last - offset][first]
            m[last - offset][first] = m[last][last - offset]
            m[last][last - offset] = m[i][last]
            m[i][last] = top
```

Time: $O(n^2)$ · Space: $O(1)$

Part XVI — Uber-Specific Patterns

The problems below come up disproportionately often in publicly reported Uber screens and onsite loops. Treat them as canon — the patterns also generalize to ride-hailing, marketplaces, and rate-limiting work.

U1. LRU Cache — LC 146 · Medium

Optimal — hash map + doubly linked list

```
class Node:
    __slots__ = ('key', 'val', 'prev', 'next')
    def __init__(self, k, v):
        self.key = k; self.val = v; self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.head = Node(0, 0); self.tail = Node(0, 0)
        self.head.next = self.tail; self.tail.prev = self.head
    def _remove(self, node):
        node.prev.next = node.next; node.next.prev = node.prev
    def _add_front(self, node):
        node.next = self.head.next; node.prev = self.head
        self.head.next.prev = node; self.head.next = node
    def get(self, key):
        if key not in self.map: return -1
        node = self.map[key]
        self._remove(node); self._add_front(node)
        return node.val
    def put(self, key, value):
        if key in self.map: self._remove(self.map[key])
        node = Node(key, value)
        self.map[key] = node; self._add_front(node)
        if len(self.map) > self.cap:
            lru = self.tail.prev
            self._remove(lru); del self.map[lru.key]
```

get/put: O(1) · Space: O(capacity)

Variations

LC 460 — LFU Cache. Frequency map + ordered buckets per frequency + min-frequency tracker.

```

from collections import OrderedDict, defaultdict
class LFUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.kv = {}
        self.freq = {}
        self.buckets = defaultdict(OrderedDict)
        self.min_f = 0
    def _bump(self, key):
        f = self.freq[key]
        v = self.buckets[f].pop(key)
        if not self.buckets[f]:
            del self.buckets[f]
            if self.min_f == f: self.min_f += 1
        self.freq[key] = f + 1
        self.buckets[f + 1][key] = v
    def get(self, key):
        if key not in self.kv: return -1
        self._bump(key)
        return self.kv[key]
    def put(self, key, value):
        if self.cap <= 0: return
        if key in self.kv:
            self.kv[key] = value
            self._bump(key); return
        if len(self.kv) >= self.cap:
            evict_key, _ = self.buckets[self.min_f].popitem(last=False)
            del self.kv[evict_key]; del self.freq[evict_key]
        self.kv[key] = value
        self.freq[key] = 1
        self.buckets[1][key] = value
        self.min_f = 1

```

get/put: O(1) · Space: O(capacity)

Time-based cache (TTL).

```

import time
class TLCache:
    def __init__(self, ttl_seconds):
        self.ttl = ttl_seconds
        self.store = {} # key -> (value, expiry)
    def get(self, key):
        v = self.store.get(key)
        if not v: return None
        val, exp = v
        if time.time() > exp:
            del self.store[key]; return None
        return val
    def put(self, key, value):
        self.store[key] = (value, time.time() + self.ttl)

```

get/put: O(1) average · Space: O(n)

U2. Trapping Rain Water — LC 42 · Hard

Brute force

For each `i`, find `maxLeft` and `maxRight`. *Time:* $O(n^2)$ · *Space:* $O(1)$

Optimal — two pointers

```
def trap(h):
    l, r = 0, len(h) - 1
    maxL = maxR = total = 0
    while l < r:
        if h[l] < h[r]:
            if h[l] >= maxL: maxL = h[l]
            else: total += maxL - h[l]
            l += 1
        else:
            if h[r] >= maxR: maxR = h[r]
            else: total += maxR - h[r]
            r -= 1
    return total
```

Time: $O(n)$ · *Space:* $O(1)$

Variations

Stack version. Each bar pops bars to the left of it forming "valleys".

```
def trap_stack(h):
    stack = []; total = 0
    for i, x in enumerate(h):
        while stack and h[stack[-1]] < x:
            top = stack.pop()
            if not stack: break
            l = stack[-1]
            total += (min(h[l], x) - h[top]) * (i - l - 1)
        stack.append(i)
    return total
```

Time: $O(n)$ · *Space:* $O(n)$

LC 407 — Trapping rain water II (3D). Min-heap from boundary inward; raise water level.

```

import heapq
def trap_rain_water_ii(heightMap):
    if not heightMap: return 0
    R, C = len(heightMap), len(heightMap[0])
    seen = [[False]*C for _ in range(R)]
    heap = []
    for r in range(R):
        for c in range(C):
            if r in (0, R-1) or c in (0, C-1):
                heapq.heappush(heap, (heightMap[r][c], r, c))
                seen[r][c] = True
    total = 0
    while heap:
        h, r, c = heapq.heappop(heap)
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r+dr, c+dc
            if 0<=nr<R and 0<=nc<C and not seen[nr][nc]:
                seen[nr][nc] = True
                total += max(0, h - heightMap[nr][nc])
                heapq.heappush(heap, (max(h, heightMap[nr][nc]), nr, nc))
    return total

```

Time: $O(MN \log MN)$ · *Space:* $O(MN)$

U3. Sliding Window Maximum — LC 239 · Hard

Brute force

For each window scan k. *Time:* $O(nk)$ · *Space:* $O(1)$

Optimal — monotonic deque

```

from collections import deque
def max_sliding_window(nums, k):
    out = []; dq = deque()
    for i, x in enumerate(nums):
        if dq and dq[0] <= i - k: dq.popleft()
        while dq and nums[dq[-1]] <= x: dq.pop()
        dq.append(i)
        if i >= k - 1: out.append(nums[dq[0]])
    return out

```

Time: $O(n)$ · *Space:* $O(k)$

Variations

LC 1438 — Longest subarray with abs diff $\leq$ limit. Two deques (max and min).

```
def longest_subarray_limit(nums, limit):
    max_dq = deque(); min_dq = deque()
    l = best = 0
    for r, x in enumerate(nums):
        while max_dq and nums[max_dq[-1]] <= x: max_dq.pop()
        while min_dq and nums[min_dq[-1]] >= x: min_dq.pop()
        max_dq.append(r); min_dq.append(r)
        while nums[max_dq[0]] - nums[min_dq[0]] > limit:
            l += 1
            if max_dq[0] < l: max_dq.popleft()
            if min_dq[0] < l: min_dq.popleft()
        best = max(best, r - l + 1)
    return best
```

Time: $O(n)$ · Space: $O(n)$

LC 862 — Shortest subarray with sum $\geq K$. Monotonic deque on prefix sums.

```
def shortest_subarray_sum_at_least_k(nums, k):
    n = len(nums); pre = [0]*(n+1)
    for i, x in enumerate(nums): pre[i+1] = pre[i] + x
    dq = deque(); best = n + 1
    for i, x in enumerate(pre):
        while dq and x - pre[dq[0]] >= k:
            best = min(best, i - dq.popleft())
        while dq and pre[dq[-1]] >= x: dq.pop()
        dq.append(i)
    return best if best <= n else -1
```

Time: $O(n)$ · Space: $O(n)$

U4. Asteroid Collision — LC 735 · Medium

Optimal — stack

```
def asteroid_collision(asteroids):
    stack = []
    for a in asteroids:
        alive = True
        while alive and a < 0 and stack and stack[-1] > 0:
            top = stack[-1]
            if top < -a: stack.pop()
            elif top == -a: stack.pop(); alive = False
            else: alive = False
        if alive: stack.append(a)
    return stack
```

Time: $O(n)$ · Space: $O(n)$

Variations

LC 946 — Validate stack sequences. Push pushed in order, pop matching popped.

```
def validate_stack_sequences(pushed, popped):
    stack = []; j = 0
    for x in pushed:
        stack.append(x)
        while stack and j < len(popped) and stack[-1] == popped[j]:
            stack.pop(); j += 1
    return j == len(popped)
```

Time: $O(n)$ · Space: $O(n)$

Variation — masses combine instead of annihilating. Slight rule tweak:

```
def asteroid_combine(asteroids):
    stack = []
    for a in asteroids:
        while stack and a < 0 and stack[-1] > 0:
            stack[-1] = stack[-1] + a # combine
            a = 0
        if stack[-1] == 0: stack.pop(); break
        if stack[-1] < 0: a = stack.pop() # consumed direction flips
    if a: stack.append(a)
    return stack
```

Time: $O(n)$ · Space: $O(n)$

U5. Random Pick with Weight — LC 528 · Medium

Optimal — prefix sums + binary search

```
import random
from bisect import bisect_left

class Solution:
    def __init__(self, w):
        self.cum = []; s = 0
        for x in w:
            s += x; self.cum.append(s)
        self.total = s
    def pickIndex(self):
        r = random.randint(1, self.total)
        return bisect_left(self.cum, r)
```

init: $O(n)$ · pick: $O(\log n)$ · Space: $O(n)$

Variations

LC 382 — Random pick from a stream (reservoir sampling).

```

class StreamPicker:
    def __init__(self, head):
        self.head = head
    def getRandom(self):
        result = None; node = self.head; i = 0
        while node:
            i += 1
            if random.randint(1, i) == 1: result = node.val
            node = node.next
        return result

```

Time: $O(n)$ per call · Space: $O(1)$

LC 398 — Random pick index by value (uniform among matches).

```

class IndexPicker:
    def __init__(self, nums): self.nums = nums
    def pick(self, target):
        result = -1; count = 0
        for i, x in enumerate(self.nums):
            if x == target:
                count += 1
                if random.randint(1, count) == 1: result = i
        return result

```

Time: $O(n)$ per call · Space: $O(1)$

LC 710 — Random pick non-blacklisted index. Map blacklisted indices in $[0..N-\text{len}(B))$ to safe indices outside the range.

```

class NonBlacklistPicker:
    def __init__(self, n, blacklist):
        self.bound = n - len(blacklist)
        bl = set(blacklist)
        next_safe = self.bound
        self.remap = {}
        for b in blacklist:
            if b < self.bound:
                while next_safe in bl: next_safe += 1
                self.remap[b] = next_safe; next_safe += 1
    def pick(self):
        i = random.randint(0, self.bound - 1)
        return self.remap.get(i, i)

```

pick: $O(1)$ · Space: $O(B)$

U6. Word Ladder — LC 127 · Hard

Optimal — BFS over wildcard buckets

```
from collections import deque, defaultdict
def ladder_length(beginWord, endWord, wordList):
    words = set(wordList)
    if endWord not in words: return 0
    L = len(beginWord)
    buckets = defaultdict(list)
    for w in words:
        for i in range(L):
            buckets[w[:i] + '*' + w[i+1:]].append(w)
    q = deque([(beginWord, 1)]); seen = {beginWord}
    while q:
        w, d = q.popleft()
        if w == endWord: return d
        for i in range(L):
            key = w[:i] + '*' + w[i+1:]
            for nb in buckets[key]:
                if nb not in seen:
                    seen.add(nb); q.append((nb, d + 1))
            buckets[key] = []
    return 0
```

Time: $O(N \cdot L^2)$ · *Space:* $O(N \cdot L^2)$

Variations

LC 126 — All shortest paths. BFS to build a parent-graph, then DFS from end backwards.

```

def find_ladders(beginWord, endWord, wordList):
    words = set(wordList)
    if endWord not in words: return []
    L = len(beginWord)
    parents = defaultdict(set)
    cur_level = {beginWord}
    found = False
    while cur_level and not found:
        next_level = defaultdict(set)
        for w in cur_level:
            for i in range(L):
                for c in 'abcdefghijklmnopqrstuvwxyz':
                    nb = w[:i] + c + w[i+1:]
                    if nb in words and nb not in parents:
                        next_level[nb].add(w)
                        if nb == endWord: found = True
        for child, ps in next_level.items():
            parents[child] |= ps
        cur_level = set(next_level.keys()) - set(parents.keys()) | set(next_level.keys())
        # remove already-seen so BFS stays leveled
        for w in next_level: words.discard(w)
    out = []
    def back(w, path):
        if w == beginWord: out.append([beginWord] + path[::-1]); return
        for p in parents[w]:
            back(p, path + [w])
    if found: back(endWord, [])
    return out

```

Time: $O(N \cdot L^2)$ · Space: $O(N \cdot L^2)$

Bidirectional BFS. Halve the worst case.

```

def ladder_length_bi(beginWord, endWord, wordList):
    words = set(wordList)
    if endWord not in words: return 0
    L = len(beginWord)
    front, back = {beginWord}, {endWord}
    seen = set([beginWord, endWord])
    steps = 1
    while front and back:
        if len(front) > len(back): front, back = back, front
        next_front = set()
        for w in front:
            for i in range(L):
                for c in 'abcdefghijklmnopqrstuvwxyz':
                    nb = w[:i] + c + w[i+1:]
                    if nb in back: return steps + 1
                    if nb in words and nb not in seen:
                        seen.add(nb); next_front.add(nb)
        front = next_front; steps += 1
    return 0

```

Time: $O(N \cdot L^2)$ · Space: $O(N)$

U7. Median of Two Sorted Arrays — LC 4 · Hard

Brute force

Merge, take middle. *Time:* $O(n)$ · *Space:* $O(n)$

Optimal — partition by binary search

```
def find_median_sorted_arrays(A, B):
    if len(A) > len(B): A, B = B, A
    n, m = len(A), len(B)
    half = (n + m + 1) // 2
    lo, hi = 0, n
    while lo <= hi:
        i = (lo + hi) // 2; j = half - i
        Ai_1 = A[i-1] if i > 0 else float('-inf')
        Ai = A[i] if i < n else float('inf')
        Bj_1 = B[j-1] if j > 0 else float('-inf')
        Bj = B[j] if j < m else float('inf')
        if Ai_1 <= Bj and Bj_1 <= Ai:
            if (n + m) % 2: return max(Ai_1, Bj_1)
            return (max(Ai_1, Bj_1) + min(Ai, Bj)) / 2
        if Ai_1 > Bj: hi = i - 1
        else: lo = i + 1
```

Time: $O(\log \min(n, m))$ · *Space:* $O(1)$

Variations

K-th element of two sorted arrays. Same partition idea targeting `k`.

```
def kth_of_two(A, B, k):
    if len(A) > len(B): A, B = B, A
    n, m = len(A), len(B)
    lo, hi = max(0, k - m), min(k, n)
    while lo <= hi:
        i = (lo + hi) // 2; j = k - i
        Ai_1 = A[i-1] if i > 0 else float('-inf')
        Ai = A[i] if i < n else float('inf')
        Bj_1 = B[j-1] if j > 0 else float('-inf')
        Bj = B[j] if j < m else float('inf')
        if Ai_1 <= Bj and Bj_1 <= Ai:
            return max(Ai_1, Bj_1)
        if Ai_1 > Bj: hi = i - 1
        else: lo = i + 1
```

Time: $O(\log \min(n, m))$ · *Space:* $O(1)$

U8. Design Hit Counter — LC 362 · Medium (LC Premium)

Optimal A — circular buffer of 300 buckets

```
class HitCounter:
    def __init__(self):
        self.times = [0] * 300
        self.hits = [0] * 300
    def hit(self, ts):
        idx = ts % 300
        if self.times[idx] != ts:
            self.times[idx] = ts; self.hits[idx] = 1
        else: self.hits[idx] += 1
    def getHits(self, ts):
        return sum(h for t, h in zip(self.times, self.hits) if ts - t < 300)
```

hit: $O(1)$ · getHits: $O(300) = O(1)$ · Space: $O(1)$

Variations

Hit counter with arbitrary window **W**. Same circular buffer of size **W**.

```
class HitCounterW:
    def __init__(self, w):
        self.w = w
        self.times = [0]*w; self.hits = [0]*w
    def hit(self, ts):
        idx = ts % self.w
        if self.times[idx] != ts:
            self.times[idx] = ts; self.hits[idx] = 1
        else: self.hits[idx] += 1
    def getHits(self, ts):
        return sum(h for t, h in zip(self.times, self.hits) if ts - t < self.w)
```

Space: $O(W)$

Deque version (variable hits per second).

```
from collections import deque
class HitCounterDeque:
    def __init__(self): self.q = deque()
    def hit(self, ts): self.q.append(ts)
    def getHits(self, ts):
        while self.q and ts - self.q[0] >= 300: self.q.popleft()
        return len(self.q)
```

hit: $O(1)$ amortized · Space: $O(\text{hits in window})$

U9. Logger Rate Limiter — LC 359 · Easy (LC Premium)

Optimal — hash map of last-seen

```
class Logger:
    def __init__(self): self.last = {}
    def shouldPrintMessage(self, ts, msg):
        if msg in self.last and ts - self.last[msg] < 10:
            return False
        self.last[msg] = ts
        return True
```

Time: $O(1)$ per call · Space: $O(\text{messages})$

Variations

Token bucket.

```
import time
class TokenBucket:
    def __init__(self, capacity, refill_rate):
        self.capacity = capacity
        self.tokens = capacity
        self.refill_rate = refill_rate # per second
        self.last = time.time()
    def allow(self):
        now = time.time()
        elapsed = now - self.last
        self.tokens = min(self.capacity, self.tokens + elapsed * self.refill_rate)
        self.last = now
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False
```

Time: $O(1)$ · Space: $O(1)$

Sliding-window log rate limiter.

```
class SlidingWindowLimiter:
    def __init__(self, limit, window_sec):
        self.limit = limit; self.window = window_sec
        self.events = defaultdict(deque) # key -> timestamps
    def allow(self, key, now):
        q = self.events[key]
        while q and now - q[0] >= self.window: q.popleft()
        if len(q) >= self.limit: return False
        q.append(now); return True
```

Time: $O(1)$ amortized · Space: $O(\text{events in window})$

U10. Surrounded Regions — LC 130 · Medium

Optimal — flood from the border

```
def solve(board):
    if not board: return
    R, C = len(board), len(board[0])
    def dfs(r, c):
        if r<0 or r>=R or c<0 or c>=C or board[r][c] != '0': return
        board[r][c] = '#'
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)): dfs(r+dr, c+dc)
    for r in range(R): dfs(r, 0); dfs(r, C-1)
    for c in range(C): dfs(0, c); dfs(R-1, c)
    for r in range(R):
        for c in range(C):
            board[r][c] = '0' if board[r][c] == '#' else 'X'
```

Time: $O(MN)$ · Space: $O(MN)$ recursion / queue

Variations

BFS variant (avoids deep recursion on big grids).

```
def solve_bfs(board):
    if not board: return
    R, C = len(board), len(board[0])
    q = deque()
    for r in range(R):
        for c in (0, C-1):
            if board[r][c] == '0': q.append((r, c)); board[r][c] = '#'
    for c in range(C):
        for r in (0, R-1):
            if board[r][c] == '0': q.append((r, c)); board[r][c] = '#'
    while q:
        r, c = q.popleft()
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r+dr, c+dc
            if 0<=nr<R and 0<=nc<C and board[nr][nc] == '0':
                board[nr][nc] = '#'; q.append((nr, nc))
    for r in range(R):
        for c in range(C):
            board[r][c] = '0' if board[r][c] == '#' else 'X'
```

Time: $O(MN)$ · Space: $O(MN)$

Union-Find with virtual border node.

```
def solve_uf(board):
    if not board: return
    R, C = len(board), len(board[0])
    dsu = DSU(R*C + 1)
    BORDER = R*C
    for r in range(R):
        for c in range(C):
            if board[r][c] != '0': continue
            if r in (0, R-1) or c in (0, C-1):
                dsu.union(r*C + c, BORDER)
            for dr, dc in ((1,0),(0,1)):
                nr, nc = r+dr, c+dc
                if 0<=nr<R and 0<=nc<C and board[nr][nc] == '0':
                    dsu.union(r*C + c, nr*C + nc)
    for r in range(R):
        for c in range(C):
            if board[r][c] == '0' and dsu.find(r*C + c) != dsu.find(BORDER):
                board[r][c] = 'X'
```

Time: $O(MN \alpha(MN))$ · Space: $O(MN)$

U11. Number of Islands II — LC 305 · Hard (LC Premium)

Optimal — Union-Find with a 4-neighbor merge

```
class DSUNbr:
    def __init__(self):
        self.p = {}
    def add(self, x):
        if x not in self.p: self.p[x] = x
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]; x = self.p[x]
        return x
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb: return False
        self.p[ra] = rb; return True

def num_islands2(m, n, positions):
    dsu = DSUNbr(); out = []; count = 0
    land = set()
    for r, c in positions:
        if (r, c) in land:
            out.append(count); continue
        land.add((r, c)); dsu.add((r, c))
        count += 1
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nb = (r+dr, c+dc)
            if nb in land:
                if dsu.union((r, c), nb): count -= 1
        out.append(count)
    return out
```

Time: $O(K \alpha(MN))$ · Space: $O(MN)$

Variations

LC 1971 — Find if path exists (online edges). Same DSU pattern.

```
def valid_path(n, edges, source, destination):
    dsu = DSU(n)
    for a, b in edges: dsu.union(a, b)
    return dsu.find(source) == dsu.find(destination)
```

Time: $O((V + E) \alpha(V))$ · *Space:* $O(V)$

Online connectivity with deletions. Hard in general; needs link-cut trees or rebuild on deletion.

U12. Find K Closest Points to Origin — LC 973 · Medium

Optimal — max-heap of size k

```
import heapq
def k_closest(points, k):
    heap = []
    for x, y in points:
        d = x*x + y*y
        if len(heap) < k:
            heapq.heappush(heap, (-d, x, y))
        elif -heap[0][0] > d:
            heapq.heapreplace(heap, (-d, x, y))
    return [[x, y] for _, x, y in heap]
```

Time: $O(n \log k)$ · *Space:* $O(k)$

Variations

Quickselect — average $O(n)$.

```

import random
def k_closest_qs(points, k):
    def dist(p): return p[0]*p[0] + p[1]*p[1]
    def part(l, r):
        pivot_d = dist(points[r])
        store = l
        for i in range(l, r):
            if dist(points[i]) < pivot_d:
                points[i], points[store] = points[store], points[i]
                store += 1
        points[store], points[r] = points[r], points[store]
        return store
    l, r = 0, len(points) - 1
    while l <= r:
        p = random.randint(l, r)
        points[p], points[r] = points[r], points[p]
        idx = part(l, r)
        if idx == k: break
        if idx < k: l = idx + 1
        else: r = idx - 1
    return points[:k]

```

Time: $O(n)$ average · Space: $O(1)$

Closest with timestamps (filter out stale points first).

```

def k_closest_fresh(points, k, max_age, now):
    fresh = [p for p in points if now - p[2] <= max_age]
    return k_closest([(x, y) for x, y, _ in fresh], k)

```

Time: $O(n \log k)$ · Space: $O(k)$

Streaming k-closest (online). Min-heap by negative distance, size k.

```

class KClosestStream:
    def __init__(self, k):
        self.k = k; self.heap = []
    def add(self, x, y):
        d = x*x + y*y
        if len(self.heap) < self.k:
            heapq.heappush(self.heap, (-d, x, y))
        elif -self.heap[0][0] > d:
            heapq.heapreplace(self.heap, (-d, x, y))
    def closest(self):
        return [[x, y] for _, x, y in self.heap]

```

add: $O(\log k)$ · Space: $O(k)$

Part XVII — Cheat sheet you can review the morning of

Time-complexity micro-budget at a glance

n	Allowed Big-O
≤ 10	$O(n!)$ backtracking is fine
≤ 20	$O(2^n)$ bitmask DP
$\leq 5,000$	$O(n^2)$ DP / nested loops
$\leq 100,000$	$O(n \log n)$ sort / heap / segment tree
$\leq 10^6$	$O(n)$ only
$\leq 10^9$	$O(\log n)$ binary search / math

"When stuck, try" checklist

1. Sort the input — does it become trivial?
2. Hash map for $O(1)$ lookup of complements / counts.
3. Two pointers / sliding window if you're scanning a contiguous range.
4. Binary search the *answer* (e.g., min capacity, min largest sum).
5. Stack / monotonic stack for "next greater" / "next smaller" patterns.
6. BFS on grids (shortest path), DFS for "all paths".
7. Dynamic programming if the same sub-problem repeats.
8. Greedy if you can prove a locally-best choice doesn't hurt globally.
9. Trie if the input is many strings sharing prefixes.
10. Heap when you only need top-k or running median.
11. Union-find when the question is "are these connected?" or "how many groups?".

Behavioral pairing — what the interviewer is also scoring

- **Communication:** narrate the trade-offs aloud.
- **Edge cases:** call them out before coding, not after the bug bites.
- **Code style:** meaningful names, small helpers, avoid clever one-liners under stress.
- **Testing:** walk through one normal case + one edge case after coding.
- **Optimization mindset:** don't dive into micro-optimizations before correctness.

Final disclaimers

- The 75 problems above approximate the *Blind 75* canon; substitutions were made for premium-only problems where freely-available equivalents exist (e.g., Encode/Decode Strings, Graph Valid Tree).
- The Uber-specific section is sourced from public interview reports on LeetCode Discuss and Glassdoor as of mid-2026. Treat them as patterns Uber tends to repeat, not guaranteed questions for any given loop.
- All code is Python 3. Mental-translate to your language of choice for interviews — most idioms (heap, deque, sorted indexing) are direct.

Good luck.